



FieldWorks

Framework Specification

Version 0.2 • Sean Slavin • July 2026

Introduction	4
What Fieldworks Is	5
What Fieldworks Is Not	7
Who This Specification Is For	8
A Note on Design Philosophy	9
Versioning	10
Version Numbering	10
Version History	10
Changelog	11
Compatibility	11
Contributing to the spec	12
Part I - Protocol Layer	14
Overview	14
Naming Conventions	15
The VQT Response Envelope	16
Error Handling	16
The Nine Required Tools	17
Conformance Requirements	22
Reference Implementations	23
What Comes Next	23
Part II - The Plant Layer	24
Overview	24
Two Files, One Deployment	24
The Topology Builder	35
Validation	39

Complete Worked Example	41
Modeling Imperfect Plants	44
What Comes Next	47
Part III - The Agent Layer	48
Overview	48
The Two Paths	48
The Agents	49
Trust Mode	55
The Audit Trail	58
The Propose/Approve/Execute Primitive	64
Part IV - The Memory Layer	67
Overview	67
The Four Stores	67
LadybugDB - The Knowledge Graph	68
DuckDB - The Analytical Layer	71
Specialist Memory Files	72
The memory-mcp server	74
The Memory Lifecycle	76
Phase 2 - Planned Memory Layer Extensions	76
The Framework Complete	78

Introduction

Industrial facilities generate continuous streams of operational data. Pumps, tanks, valves, dosing systems, UV banks. Every piece of equipment produces signals that describe its condition. The people who need to understand those signals — operators, maintenance technicians, plant managers — have always been separated from them by the tools required to access them.

For decades the answer was SCADA systems and historians. They normalized the chaos of industrial protocols, built object models from the noise and made data visible to those who knew how to read a trend screen or write a query. That was the right answer for its time. The barrier got lower. But it never went away.

The barrier today is not connectivity. The data is accessible. The barrier is fluency. You need to know which tag to query, which trend to pull, which historian to open, which dashboard to read. You need to know what normal looks like to recognize abnormal. You need to know the plant.

Fieldworks removes that barrier. Not by replacing the historians, SCADA platforms or protocol infrastructure, those already work, but by putting a natural language interface on top of them. The same way a web browser didn't replace the control system but made its value accessible to anyone with a network connection, Fieldworks makes plant intelligence accessible to anyone who can ask a question.

The interface is a chat window. The intelligence is the plant.

What Fieldworks Is

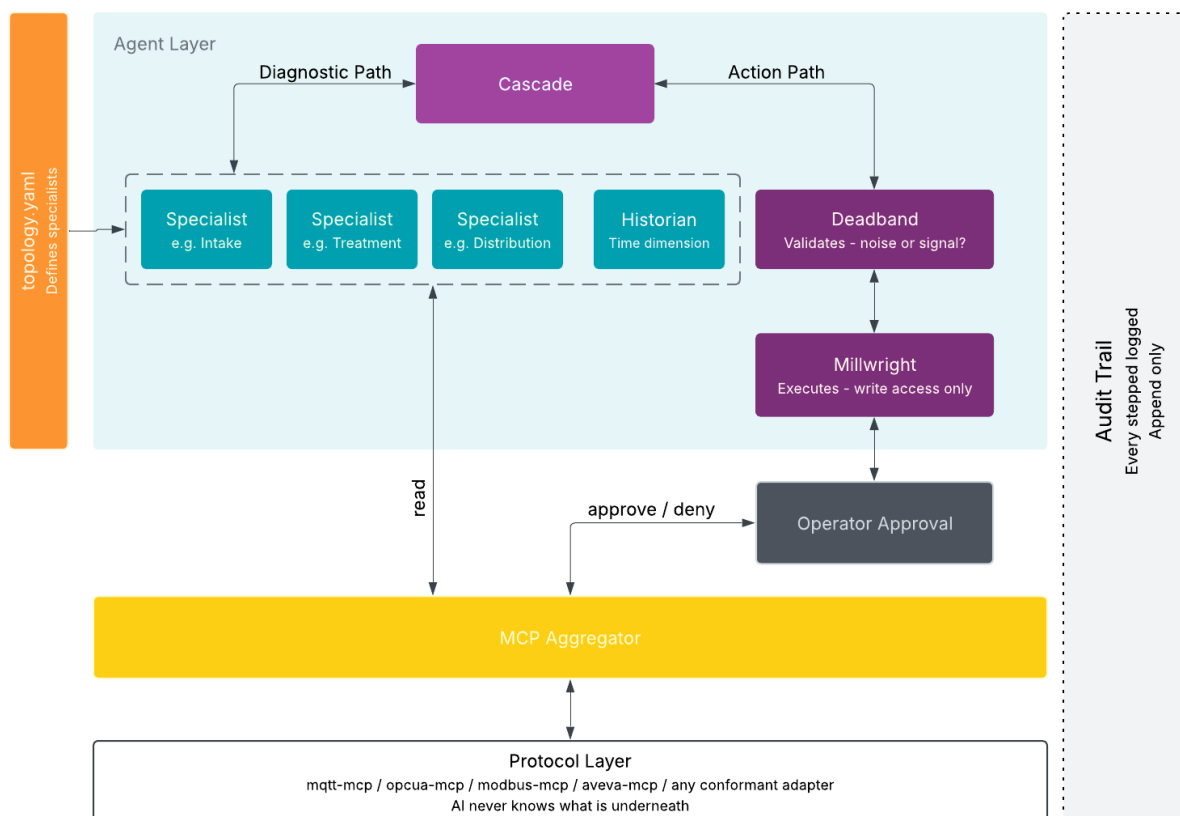
Fieldworks is an open source, MCP-native industrial AI framework. It is protocol-first, plant-agnostic and designed to run fully on-premises in air-gapped environments.

It is not a cloud service. It is not a proprietary platform. It is not a point solution for a specific protocol or vendor. It is a framework. A set of architectural patterns, configuration schemas, agent designs and protocol contracts that any OT engineering team can deploy, extend and build on.

The framework has three layers.

The protocol layer is a collection of conformant MCP servers. One per industrial protocol. MQTT, OPC-UA, Modbus, DNP3, EtherNet/IP, AVEVA System Platform. Each protocol becomes a standardized tool surface. The AI never knows what protocol is underneath. Swap the adapter, nothing else changes.

The plant layer is driven by a single configuration file, topology.yaml, that describes the facility. Equipment types, instances, attributes, normal operating ranges, fault modes, process areas and write permissions. From this one file, the framework derives specialist agent configurations, fault detection baselines, dashboard definitions and system prompt context. Changing the config changes the plant. The framework code does not change.



The agent layer is where reasoning happens. Cascade orchestrates. Specialists diagnose. Deadband validates. Millwright executes. The Historian provides time dimension across all of it. These agents work together across the diagnostic and action paths, with every step logged to an append-only audit trail.

What Fieldworks Is Not

Fieldworks is not a replacement for your control system. It does not manage setpoints autonomously without your knowledge. It does not make safety decisions. It does not operate independently of the people responsible for the plant.

It is a tool that makes those people more capable. It surfaces conditions they might have missed. It proposes actions for their consideration. It executes what they approve. It documents everything — the condition, the reasoning, the decision, the outcome — in a form that satisfies compliance requirements and builds operator trust over time.

The goal is not autonomous operation. The goal is accessible intelligence, with the operator always in the loop until they decide, consciously and deliberately, to widen the loop.

Who This Specification Is For

This specification has three audiences, corresponding to three stages of engagement with Fieldworks.

Protocol adapter builders — SIs, equipment vendors and OT engineers who want to build a conformant MCP server for a protocol not yet supported. Part I covers the protocol layer specification. Required tools, signatures, metadata, error handling, naming conventions.

Plant deployers — OT engineering teams deploying Fieldworks against a real facility. Part II covers the plant layer specification. The topology config schema, process area definitions, equipment type modeling, fault mode design, write permission configuration.

Framework contributors — teams extending or building on the agent layer. Part III covers the agent layer specification. The fixed agent designs, specialist instantiation patterns, the action path, trust modes and the audit trail contract.

These are not separate audiences. They are stages of the same journey. The adapter builder becomes the deployer becomes the contributor. The specification is written to support that progression.

A Note on Design Philosophy

Every decision in Fieldworks reflects a single conviction. The data is at the plant, the people who need it are sometimes elsewhere and the barrier between them is a technology problem worth solving.

The web browser solved this problem once. Not by replacing the control system but by making its value accessible to people who couldn't otherwise reach it. Fieldworks solves it again, for a different barrier, with different tools.

The chat window is not the point. Accessible plant intelligence is the point. The chat window is just the current best answer to the question of how you remove the barrier between an operator and their plant at 2am.

That answer will change. The problem will not.

Versioning

Current version: 0.2

July 2026 · Sean Slavin · fieldworks.build

Version Numbering

FieldWorks uses semantic versioning. Version numbers follow the pattern MAJOR.MINOR.PATCH.

- **Major version** — breaking changes to the conformant tool surface, schema changes that require adapter updates, or agent behavior changes that break existing deployments. A deployment built against v1.x must be reviewed before upgrading to v2.0.
- **Minor version** — additive changes. New optional fields, new sections, new tools that do not break existing conformant adapters. A deployment built against v0.1 will continue to work on v0.2 without modification.
- **Patch version** — corrections and clarifications. No behavioral changes. Patch releases do not require any deployment review.

v0.x versions are pre-release. The framework architecture is established but the conformant tool surface and schema details are subject to change between minor versions. Implementations should expect the surface to stabilize at v1.0. When the spec reaches v1.0, the major/minor/patch stability guarantees apply in full.

Version History

Version	Date	Summary
0.1	2026-06	Initial release. Introduction, Parts I–IV, Topology Builder, Audit Trail, Versioning sections.
0.2	2026-07	Added the Propose/Approve/Execute mechanism to Part III (SUPERVISED mode only). Documented as a new framework contract, not a description of existing behavior.

New versions will be listed here as the spec evolves. Each entry links to the changelog below for detail on what changed.

Changelog

v0.2 — July 2026

Added the Propose/Approve/Execute mechanism to Part III, extracted from the waterworks-ai reference application into a shared, framework-level contract.

- Part III — new section: The Propose/Approve/Execute Primitive. Defines the propose/hold/approve-or-deny flow, the division of responsibility between framework and host, the two-tier audit record (compliance vs. operational) and the current scope boundary (SUPERVISED mode only; ADVISORY, COLLABORATIVE, and AUTONOMOUS mode-dispatch logic remain unimplemented).
- This is an additive, non-breaking change per the versioning policy defined below.

v0.1 — June 2026

Initial publication of the FieldWorks Framework Specification.

- Introduction — framework overview, three-layer architecture, design philosophy, audience guide.
- Part I — Protocol layer specification. Nine required tools, VQT response envelope, error handling, naming conventions, conformance requirements, reference implementations.
- Part II — Plant layer specification. topology.yaml schema, aggregator.json schema, equipment type and instance modeling, fault modes, topology builder, legacy mode, validation, modeling imperfect plants.
- Part III — Agent layer specification. Five fixed agents (Cascade, Specialist, Historian, Deadband, Millwright), diagnostic and action paths, trust mode ladder, audit trail contract.
- Part IV — Memory layer specification. Four-store architecture (LadybugDB, DuckDB, InfluxDB, specialist memory files), memory-mcp server, memory lifecycle, Phase 2 planned extensions.
- Versioning — this section.

Compatibility

Spec version to implementation

The waterworks-ai reference implementation tracks the current spec version. The README documents which spec version it implements.

Version	Date	Summary
0.1	2026-06	waterworks-ai v0.1 — Phases 8–10 implemented. Protocol layer, plant layer, four-store memory, topology builder.

0.2	2026-07	fieldworks-core adds the Propose/Approve/Execute mechanism (SUPERVISED mode only). waterworks-ai has not yet migrated its existing action-path code onto it.
-----	---------	--

Breaking change policy

Breaking changes increment the major version. The following are considered breaking:

- Removing or renaming a required tool from the conformant tool surface.
- Changing the VQT response envelope schema in a way that breaks existing consumers.
- Changing required fields in topology.yaml or aggregator.json.
- Changing the LadybugDB schema in a way that requires migration of existing graphs.
- Changing agent behavior in a way that breaks existing operator workflows.

The following are not breaking and increment the minor version:

- Adding new optional fields to any schema.
- Adding new tools to the conformant surface that do not replace existing ones.
- Adding new sections or parts to the specification.
- Adding new event types to the audit trail.
- Adding new trust mode configurations.

Deprecation policy

Features deprecated in a minor version are supported for at least one full major version before removal. Deprecated features are marked in the specification with a deprecation notice and the target removal version.

No features are deprecated in v0.1.

Contributing to the spec

The FieldWorks Framework Specification is maintained in the FieldWorks GitHub organization. Proposed changes go through a review process before being incorporated.

- **Bug reports and clarifications** — open an issue in the fieldworks-spec repository. Patch releases are cut regularly to incorporate clarifications.
- **Additive proposals** — open a pull request with the proposed text. Additive changes that do not break existing implementations are incorporated into the next minor release.
- **Breaking change proposals** — open an issue for discussion before writing spec text. Breaking changes require a clear migration path and are batched into major releases.

The reference implementations are authoritative when the spec is ambiguous. If a reference implementation and the spec disagree, the spec takes precedence and the implementation is a bug — unless the spec is genuinely unclear, in which case a clarification patch is issued first.

Part I - Protocol Layer

Overview

The protocol layer is the foundation of FieldWorks. It is a collection of conformant MCP servers, one per industrial protocol, that give the agent layer a standardized, consistent tool surface regardless of what protocol is running underneath.

Every conformant FieldWorks MCP server exposes the same nine tools. Seven tool names are identical across all adapters. Two tools, incremental address space exploration and full tree retrieval, use protocol-appropriate names that reflect the fundamentally different discovery models of each protocol. The input signatures and response envelope are identical for the seven universal tools.

For the two protocol-specific slots, the response shape is identical but the names and parameters differ per protocol. The AI never knows whether it is reading from an MQTT broker, an OPC-UA server or a PI historian. It calls the same tools with the same parameters and receives the same shape of response.

Conceptual Slot	MQTT	OPCUA
Incremental exploration	scan	browse
Full tree retrieval	get_topic_tree	get_node_tree

Swap the adapter. Nothing else changes.

This part of the specification is written for protocol adapter builders. System integrators, equipment vendors and OT engineers who want to build a conformant MCP server for a protocol not yet covered by the framework. It is self-contained. You do not need to read Parts II or III to implement a conformant adapter.

The reference implementations are `mqtt-mcp` and `opcua-mcp`, both available in the FieldWorks GitHub organization. When this specification is ambiguous, those implementations are authoritative.

Naming Conventions

Server naming

MCP servers are named using the pattern:

```
{protocol}-mcp
```

Examples: mqtt-mcp, opcua-mcp, modbus-mcp, dnp3-mcp, ethernetip-mcp, aveva-mcp.

The server name is the protocol namespace. It provides context for every tool call. A tool called `read_tag` on `mqtt-mcp` is unambiguously `mqtt-mcp::read_tag`. Tool names carry no protocol prefix. The server name provides it.

Tool naming

Tools use `snake_case verb_noun` format. No protocol prefix. No server prefix. The verb describes the action. The noun describes the target.

```
{verb}_{noun}
```

The nine required tools follow this convention without exception. Protocol-specific extensions, if any, must also follow it.

Parameter naming

Parameters use `snake_case` throughout. Tag identifiers are passed as `tag_id`. Time boundaries use `start_time` and `end_time`. Filter parameters use `process_area` and `equipment_id` consistently across all tools that accept them.

Implementation note - Rust trait

The Rust adapter trait in `fieldworks-adapters` uses the MQTT tool names as canonical for the two protocol-specific slots: `scan` and `get_topic_tree`. OPC-UA adapters rename these to `browse` and `get_node_tree` in their implementations. Adapters targeting other protocols follow

the same pattern. Choose the protocol-appropriate name and document the mapping in the adapter README.

The VQT Response Envelope

VQT — Value, Quality, Timestamp — is the gold standard of industrial data. Every historian, every SCADA platform, every process data infrastructure speaks it. FieldWorks codifies it as the required response envelope for all data operations.

Every tool that returns tag data — `read_tag`, `read_tag_history`, `write_tag` confirmation — returns the same envelope:

```
{
  "tag_id": "string",
  "value": "number | string | boolean",
  "quality": "good | uncertain | bad",
  "timestamp": "ISO 8601 string",
  "units": "string"
}
```

Field definitions

- **tag_id** — The tag identifier as passed in the request. Echoed back for correlation.
- **value** — The tag value. Type varies by tag data type. Numeric process values are float. Boolean states are boolean. String descriptors are string.
- **quality** — One of three states: good (value is reliable and current), uncertain (value may be stale, interpolated or from a degraded source), bad (value is not reliable; do not use for decisions)
- **timestamp** — ISO 8601 UTC. Always UTC. Never local time. Millisecond precision preferred.
- **units** — Engineering units string. Examples: degC, bar, m3/h, %, NTU, mg/L. Empty string if unitless or not applicable.

Quality is not optional. If the underlying protocol does not provide quality information natively, the adapter must infer it. A value received from a disconnected or timed-out source is bad. A value that exceeds the configured engineering range is uncertain. A value received normally from a healthy source is good.

Error Handling

All tools return structured errors. Adapters must never throw unhandled exceptions that surface raw protocol errors to the agent layer. All protocol-specific errors are caught, translated, and returned in the standard error envelope.


```
{
  "error": {
    "code": "string",
    "message": "string",
    "tag_id": "string | null"
  }
}
```

Required error codes

- **TAG_NOT_FOUND** — The requested tag_id does not exist on this server.
- **TAG_NOT_WRITABLE** — A write was attempted on a tag not configured for write access.
- **CONNECTION_ERROR** — The adapter cannot reach the protocol endpoint.
- **TIMEOUT** — The protocol endpoint did not respond within the configured timeout window.
- **INVALID_VALUE** — The value provided to write_tag is outside the configured engineering range or wrong type.
- **PERMISSION_DENIED** — The write was blocked by topology configuration write permissions.
- **HISTORY_UNAVAILABLE** — read_tag_history was called but the protocol or endpoint does not support historical data.

Error messages must be human-readable. They will surface to operators. Protocol-specific codes and addresses should not appear in the message field.

The Nine Required Tools

Every conformant FieldWorks MCP server implements exactly these nine tools. No more are required. Additional protocol-specific tools may be added but do not replace these. The agent layer depends on this surface being consistent and complete.

Lifecycle Tools

connect

Establish a connection to the protocol endpoint. Called once on startup and on reconnect after a connection failure. Returns connection status and server metadata.

Tool	connect
Description	Establish connection to the protocol endpoint.

Parameters	host (string, required) — Hostname or IP address of the protocol endpoint. port (integer, required) — Port number. timeout_ms (integer, optional) — Connection timeout in milliseconds. Default 5000. options (object, optional) — Protocol-specific connection options (credentials, TLS config, client ID, etc.).
Returns	Connection status object: { connected: boolean, server_name: string, protocol_version: string, timestamp: ISO 8601 }

disconnect

Cleanly terminate the connection to the protocol endpoint. Adapters must flush any pending operations before disconnecting. Called on graceful shutdown.

Tool	disconnect
Description	Cleanly terminate the connection to the protocol endpoint.
Parameters	reason (string, optional) — Human-readable reason for disconnection. Logged to audit trail.
Returns	{ disconnected: boolean, timestamp: ISO 8601 }

Discovery Tools

discover_tags

Enumerate available tags with full metadata. The primary mechanism for the plant layer to understand what data is available on a given adapter. Supports optional filtering by process area or equipment instance.

Tool	discover_tags
Description	Enumerate available tags with metadata. Returns the full tag inventory or a filtered subset.
Parameters	process_area (string, optional) — Filter results to tags belonging to this process area. Must match a process_area defined in topology.yaml. equipment_id (string, optional) — Filter results to tags belonging to this equipment instance. include_metadata (boolean, optional) — Include extended metadata (description, normal range, fault modes). Default true.

Returns	Array of tag descriptors: [{ tag_id, name, description, units, data_type, writable, process_area, equipment_id, normal_range: { min, max } }]
----------------	---

browse (OPC-UA) / scan (MQTT)

browse and scan occupy a single conformant slot. Incremental address space exploration. A conformant adapter implements one, not both, based on its protocol. These tools share a conceptual purpose but reflect the fundamentally different discovery models of their respective protocols.

browse (OPC-UA) traverses the OPC-UA address space hierarchically from a given node. The address space is a tree; browse navigates it. Returns child nodes with their NodeId, BrowseName, NodeClass and display name. Recurse by calling browse on returned node IDs.

scan (MQTT) discovers active topics by subscribing to a wildcard pattern and collecting published topics over a defined observation window. Returns the set of topics seen during the window with their last known value and quality.

Tool	browse / scan
Description	Protocol-native address space exploration.
Parameters	node_id (string, optional) — (OPC-UA) Starting node for traversal. Defaults to root Objects node. topic_pattern (string, optional) — (MQTT) Wildcard subscription pattern. Default #. scan_duration_ms (integer, optional) — (MQTT) Observation window duration. Default 5000. depth (integer, optional) — (OPC-UA) Maximum traversal depth. Default 3. Use -1 for unlimited.
Returns	Array of discovered nodes or topics with available metadata.

get_topic_tree / get_node_tree

get_topic_tree and get_node_tree occupy a single conformant slot. Full tree retrieval. A conformant adapter implements one, not both, based on its protocol. Where browse/scan is an exploratory tool for navigating incrementally, get_topic_tree and get_node_tree return the full picture at once.

This tool is intentionally expensive. It is designed for two use cases: initial topology.yaml construction when onboarding a new plant, and periodic refresh to detect newly added equipment or sensors. It should not be called at query runtime.

Tool	get_topic_tree / get_node_tree
Description	Return the complete address space as a structured hierarchical object.

Parameters	root_node (string, optional) — (OPC-UA) Root node for tree traversal. Default: Objects root. topic_prefix (string, optional) — (MQTT) Prefix to scope the tree. Default: empty (full tree). include_values (boolean, optional) — Include current values for leaf nodes. Increases response size significantly. Default false.
Returns	Nested object representing the full address space hierarchy. Leaf nodes include tag_id, name, data_type, units, and optionally current VQT.

Data Tools

read_tag

Read the current value of a single tag. The most frequently called tool in the framework. Returns the VQT envelope. Quality must always be present.

Tool	read_tag
Description	Read the current value of a single tag by identifier.
Parameters	tag_id (string, required) — Tag identifier as returned by discover_tags or browse/scan.
Returns	VQT envelope: { tag_id, value, quality, timestamp, units }

read_tag_history

Read a time series for a tag over a defined window. Used by the Historian agent to provide time dimension to diagnostic queries. Returns an array of VQT envelopes ordered by timestamp ascending.

Not all protocols support historical data natively. Adapters for protocols without native historian capability must return a HISTORY_UNAVAILABLE error rather than attempting to simulate history.

Tool	read_tag_history
Description	Read a time series for a tag over a defined time window.

Parameters	tag_id (string, required) — Tag identifier. start_time (string, required) — Window start. ISO 8601 UTC. end_time (string, required) — Window end. ISO 8601 UTC. max_points (integer, optional) — Maximum number of data points to return. Adapter may interpolate or downsample to fit. Default 1000. quality_filter (string, optional) — Return only values meeting this quality threshold. One of: good, good_or_uncertain. Default: no filter.
Returns	Array of VQT envelopes ordered by timestamp ascending: [{ tag_id, value, quality, timestamp, units }]

write_tag

Write a value to a tag. The only tool in the protocol layer with side effects on the physical process. Write access is gated by the topology configuration. An adapter must refuse writes to any tag not explicitly configured as writable in topology.yaml.

Millwright is the only agent that calls write_tag. The topology configuration defines which tags are writable and the hard limits that bound any write value. These limits cannot be overridden by any agent, including Millwright, regardless of what the diagnostic path has determined.

Tool	write_tag
Description	Write a value to a tag. Gated by topology write permissions.
Parameters	tag_id (string, required) — Tag identifier. Must appear in topology write_permissions to succeed. value (number boolean, required) — Value to write. Must be within the configured engineering range. units (string, required) — Engineering units of the provided value. Adapter validates against configured units. operator_id (string, required) — Identifier of the operator who approved this write. Logged to audit trail. reason (string, required) — Human-readable reason for the write. Logged to audit trail.
Returns	Write confirmation VQT: { tag_id, value_written, quality, timestamp, units, operator_id }

operator_id and reason are required even when the write originates from an automated action path. The topology configuration determines what operator approval means in each deployment context — it may be a named individual, a role, or a session token. The adapter logs it; the topology defines it.

Server Tool

get_server_info

Returns metadata about the adapter and its connection state. Called by Cascade on initialization to understand what protocol surfaces are available and their current health. Also useful for monitoring and diagnostics.

Tool	get_server_info
Description	Return metadata about this adapter and its current connection state.
Parameters	None
Returns	{ server_name: string, protocol: string, protocol_version: string, connected: boolean, connection_state: string, capabilities: [string], uptime_seconds: integer, last_error: string null }

capabilities is an array of tool names the server supports. All nine required tools are always listed. Protocol-specific extensions appear here as well. Cascade uses this to understand what it can rely on from each adapter.

Conformance Requirements

A FieldWorks-conformant MCP server meets all of the following requirements:

- Implements all nine required tools with the exact signatures defined in this specification.
- Returns the VQT envelope for all data operations. Quality is never absent.
- Returns structured errors in the standard error envelope. Raw protocol errors never surface to the agent layer.
- Enforces topology write permissions on write_tag. Writes to tags not in topology write_permissions return TAG_NOT_WRITABLE.
- Logs all write operations with operator_id and reason. The log is append-only.
- Reports accurate capabilities in get_server_info.
- Uses UTC timestamps in ISO 8601 format throughout.
- Follows the naming conventions defined in this specification.

Adapters that meet these requirements are conformant. Adapters that do not are not FieldWorks adapters, regardless of whether they implement some subset of the tools.

Partial conformance is not conformance. The agent layer assumes the full surface is present. An adapter missing read_tag_history will cause Historian to fail silently. An adapter that surfaces raw protocol errors will confuse operators. The requirements exist because the framework depends on them.

Reference Implementations

mqtt-mcp and **opcua-mcp** are the reference implementations of this specification. Both are open source and available in the FieldWorks GitHub organization.

When this specification is ambiguous, the reference implementations are authoritative. When the reference implementations diverge from this specification, this specification takes precedence and the implementation is a bug.

mqtt-mcp is the simpler of the two. It is the recommended starting point for understanding the conformant surface. The demo mode, deterministic synthetic data without a live broker, is particularly useful for understanding how the tool surface behaves under normal and fault conditions without requiring protocol infrastructure.

opcua-mcp demonstrates the browse/get_node_tree pattern and the OPC-UA address space model. The node hierarchy traversal and NodeId handling in opcua-mcp are the reference for any adapter targeting a protocol with a structured address space.

What Comes Next

Part I is complete when you have a conformant MCP server that passes the conformance requirements. At that point you have a protocol adapter that can plug into any FieldWorks deployment.

Part II covers the plant layer. How to describe a real facility in topology.yaml so the agent layer understands the equipment, process areas, normal operating ranges, fault modes and write permissions. Part II assumes you have conformant adapters underneath.

Part III covers the agent layer. Cascade, the Specialists, Deadband, Millwright, the Historian, the action path and the audit trail contract. Part III assumes Parts I and II.

Part II - The Plant Layer

Overview

The plant layer is where FieldWorks learns the facility. Protocol adapters give the framework access to data. The plant layer gives that data meaning.

Two configuration files define the plant layer. Together they tell the framework what the facility is, how it is structured, what normal looks like, what failure looks like and how the protocol surface maps onto the physical world.

- **topology.yaml** describes the facility. Its process areas, the types of equipment deployed, the specific instances of that equipment, the tag IDs that represent each attribute, normal operating ranges and fault mode descriptions. This is the plant model.
- **aggregator.json** describes the deployment. Which protocol servers are available, which tools each server exposes and how each server is scoped to a section of the protocol surface. This is the connection model.

Part II is written for plant deployers. OT engineering teams configuring FieldWorks against a real facility. It assumes you have conformant protocol adapters in place. If you are building an adapter, read Part I first.

The reference scenario throughout this part is waterworks-ai, a water treatment plant deployment available as the FieldWorks reference implementation.

Two Files, One Deployment

The separation between topology.yaml and aggregator.json is deliberate. They answer different questions.

topology.yaml answers, what is this plant? It describes physical reality. A centrifugal pump is a centrifugal pump regardless of whether it reports via MQTT or OPC-UA. The equipment type, its attributes, its normal operating ranges and its failure modes are properties of the physical equipment, not the protocol it happens to speak. topology.yaml captures that physical knowledge.

aggregator.json answers, how do we connect to it? It describes the protocol surface and how that surface is presented to the agent layer. The same MQTT broker might be split into multiple logical server instances, each scoped to a different section of the topic tree. The aggregator config is where those deployment decisions live.

Keeping them separate means you can change the connection topology without touching the plant model and vice versa. You can also reuse equipment type definitions across facilities with different protocol configurations.

topology.yaml

topology.yaml is a single YAML file that fully describes the facility being monitored. It has five top-level sections.

```
facility:
  # Facility metadata

process_areas:
  # Named zones mapping to Specialist agents

equipment_types:
  # Class definitions for equipment categories

equipment_instances:
  # Deployed equipment with tag ID bindings

historian:
  # Time dimension configuration
```

facility

Top-level metadata about the facility. Used by Cascade to provide context in responses and by the audit trail for log attribution.

Key	Type	Required	Description
name	string	yes	Human-readable facility name. Appears in agent responses and audit log headers.
site_id	string	yes	Unique identifier for this site. Used for audit trail attribution and multi-site deployments.
description	string	no	Brief description of the facility type and function. Provides Cascade with context for response framing.
timezone	string	yes	IANA timezone string. e.g. America/New_York. Used for local time display in operator-facing responses.

<code>units_system</code>	string	no	Preferred units system: metric or imperial. Default: metric. Individual attributes override this.
---------------------------	--------	----	---

facility:

```

name: Riverside Water Treatment Plant
site_id: wtp-riverside-01
description: Municipal water treatment facility, 45 MGD capacity
timezone: America/Chicago
units_system: metric

```

process_areas

Process areas divide the facility into named zones. Each process area maps directly to a Specialist agent. When Cascade receives a query, it determines which process areas are relevant and dispatches the corresponding Specialists.

Process area names should reflect how operators naturally refer to sections of the plant. They appear in agent responses and in the audit trail.

Key	Type	Required	Description
<code>id</code>	string	yes	Unique identifier for this process area. Referenced by equipment instances.
<code>name</code>	string	yes	Human-readable name. Used in operator-facing responses.
<code>description</code>	string	yes	What this area does. Provides the Specialist with context for its diagnostic role.
<code>specialist_prompt</code>	string	no	Additional context injected into the Specialist system prompt for this area. Use for facility-specific knowledge the Specialist should always have.

```

process_areas:
  - id: intake
    name: Raw Water Intake
    description: Screens and pumps raw water from the source into the
      treatment process. Primary equipment is raw water pumps and intake screens.
    specialist_prompt: This plant draws from a river source with seasonal
      turbidity variation. High turbidity events are common in spring.

  - id: treatment
    name: Treatment
    description: Coagulation, flocculation, sedimentation, and filtration.
      Includes chemical dosing systems for coagulant and pH adjustment.

  - id: distribution
    name: Treated Water Distribution
    description: Pumping of treated water to the distribution network.
      Includes high-service pumps and clear well level management.

```

equipment_types

Equipment types are class definitions. They define the attribute schema, normal operating ranges and fault mode descriptions that apply to every instance of that equipment category. This is the object-oriented foundation of the topology model.

A real plant may have dozens of centrifugal pumps, each with identical attribute schemas and similar fault modes. Define the type once. Instances reference it and bind their specific tag IDs to the attribute names.

Key	Type	Required	Description
<code>id</code>	string	yes	Unique identifier for this equipment type. Referenced by instances.
<code>name</code>	string	yes	Human-readable type name. e.g. Centrifugal Pump, UV Disinfection Bank.
<code>description</code>	string	yes	What this type of equipment does in the process. Provides Specialists with equipment context.
<code>attributes</code>	list	yes	List of attribute definitions. See Attributes below.
<code>fault_modes</code>	list	yes	List of fault mode descriptions. See Fault Modes below.

```

equipment_types:
  - id: centrifugal_pump
    name: Centrifugal Pump
    description: Moves water through the process via impeller rotation.
      Primary equipment in intake and distribution areas.
    attributes:
      - id: discharge_pressure
        name: Discharge Pressure
        units: bar
        normal_range:
          min: 2.8
          max: 4.2
          description: Varies with system demand. Lower end during low-demand
periods.
      - id: motor_current
        name: Motor Current
        units: A
        normal_range:
          min: 18.0
          max: 32.0
      - id: vibration
        name: Vibration
        units: mm/s
        normal_range:
          min: 0.0
          max: 4.5
      - id: bearing_temp
        name: Bearing Temperature
        units: degC
        normal_range:
          min: 20.0
          max: 75.0
      - id: run_status
        name: Run Status
        units: ""
        normal_range:
          min: 0
          max: 1
      - id: speed_setpoint
        name: Speed Setpoint
        description: VFD speed setpoint as percentage of rated speed.
        units: "%"
        normal_range:
          min: 40.0
          max: 100.0
        writable: true

```

Continued on the next page.

```
fault_modes:
- id: bearing_wear
  name: Bearing Wear
  severity: warning
  affected_attributes: [vibration, bearing_temp, motor_current]
  description: Bearing wear presents as a gradual increase in vibration
    over hours to days, typically starting on one axis before spreading.
    Bearing temperature rises as friction increases. Motor current may
    increase slightly as mechanical resistance grows. Early stage is
    often detectable only in trend data. Advanced bearing wear can
    progress to failure rapidly. Common causes are inadequate lubrication,
    contamination, or end-of-service-life.
- id: cavitation
  name: Cavitation
  severity: warning
  affected_attributes: [discharge_pressure, vibration, motor_current]
  description: Cavitation presents as erratic discharge pressure with
    high-frequency vibration. Characteristically noisy in the field.
    Motor current becomes unstable. Sustained cavitation causes rapid
    impeller damage. Common causes are insufficient suction head, high
    flow demand exceeding pump curve, or air entrainment in suction.
- id: seal_failure
  name: Seal Failure
  severity: critical
  affected_attributes: [discharge_pressure, motor_current]
  description: Seal failure presents as a sudden drop in discharge
    pressure accompanied by a drop in motor current as the pump loses
    prime. In wet well installations this is accompanied by visible
    leakage. Requires immediate shutdown. Do not restart without
    seal inspection and replacement.
```

Attributes

Attributes are the measurable [properties](#) of an equipment type. Each attribute defines what is being measured, its engineering units, and its normal operating range. Instances bind tag IDs to these attribute names.

Key	Type	Required	Description
id	string	yes	Attribute identifier. Referenced in equipment instances to bind tag IDs.
name	string	yes	Human-readable attribute name. e.g. Discharge Pressure, Motor Current.
description	string	no	What this attribute measures and why it matters diagnostically.
units	string	yes	Engineering units. e.g. bar, degC, A, m3/h, NTU, mg/L.
normal_range.min	number	yes	Lower bound of normal operating range.
normal_range.max	number	yes	Upper bound of normal operating range.
normal_range.description	string	no	Narrative context for the normal range. Useful for ranges that vary by operating mode.
writable	boolean	no	Whether this attribute accepts writes via <code>write_tag</code> . Default false. Write permissions are enforced at the aggregator layer.

Fault Modes

Fault modes describe failure patterns the Specialist should recognize. They are natural language descriptions, not structured rules. The Specialist reasons against them using its understanding of the equipment, the current tag values and the historical trend.

Write fault mode descriptions the way an experienced maintenance technician would brief a new engineer. Include what to look for, what other symptoms typically accompany it and what the likely cause is. The richer the description, the more useful the Specialist.

Key	Type	Required	Description
id	string	yes	Unique identifier for this fault mode within the equipment type.
name	string	yes	Short name for this fault mode. e.g. Bearing Wear, Cavitation, Seal Failure.

<code>description</code>	string	yes	Natural language description of how this fault presents, what attributes are affected, typical progression, and likely causes. No length limit.
<code>severity</code>	string	yes	One of: advisory, warning, critical. Guides Deadband threshold and escalation behavior.
<code>affected_attributes</code>	list	yes	List of attribute IDs that are diagnostic indicators for this fault mode.

equipment_instances

Equipment instances are the deployed equipment in the facility. Each instance references an equipment type and binds the type's attribute IDs to specific tag IDs on the protocol surface. This is where the abstract type model connects to the real plant.

Key	Type	Required	Description
<code>id</code>	string	yes	Unique identifier for this equipment instance. Referenced in operator queries and the audit trail.
<code>name</code>	string	yes	Human-readable instance name. e.g. Raw Water Pump 1.
<code>type_id</code>	string	yes	References an equipment type defined in <code>equipment_types</code> .
<code>process_area_id</code>	string	yes	References a process area defined in <code>process_areas</code> . Determines which Specialist is responsible.
<code>description</code>	string	no	Instance-specific notes. Use for information that differs from the type default, such as a different service history or known condition.
<code>tag_bindings</code>	map	yes	Maps attribute IDs from the equipment type to tag IDs on the protocol surface. Every attribute defined on the type must have a binding.
<code>commissioned</code>	string	no	ISO 8601 date the equipment was commissioned. Used by the Historian for long-term trend context.
<code>notes</code>	string	no	Free-form operational notes. Visible to Specialists. Use for known conditions, recent maintenance, or anything that context that affects diagnosis.

```

equipment_instances:
  - id: raw_water_pump_1
    name: Raw Water Pump 1
    type_id: centrifugal_pump
    process_area_id: intake
    commissioned: "2019-03-15"
    notes: Replaced impeller 2024-11-02 following cavitation damage.
          Running well since. Monitor suction conditions during high-demand periods.
    tag_bindings:
      discharge_pressure: Plant/WTP/Pump/RawWater_P01/DischargePressure
      motor_current:      Plant/WTP/Pump/RawWater_P01/MotorCurrent
      vibration:          Plant/WTP/Pump/RawWater_P01/Vibration
      bearing_temp:       Plant/WTP/Pump/RawWater_P01/BearingTemp
      run_status:         Plant/WTP/Pump/RawWater_P01/RunStatus
      speed_setpoint:     Plant/WTP/Pump/RawWater_P01/SpeedSetpoint

  - id: raw_water_pump_2
    name: Raw Water Pump 2
    type_id: centrifugal_pump
    process_area_id: intake
    commissioned: "2021-07-08"
    tag_bindings:
      discharge_pressure: Plant/WTP/Pump/RawWater_P02/DischargePressure
      motor_current:      Plant/WTP/Pump/RawWater_P02/MotorCurrent
      vibration:          Plant/WTP/Pump/RawWater_P02/Vibration
      bearing_temp:       Plant/WTP/Pump/RawWater_P02/BearingTemp
      run_status:         Plant/WTP/Pump/RawWater_P02/RunStatus
      speed_setpoint:     Plant/WTP/Pump/RawWater_P02/SpeedSetpoint

```

Both pumps reference the centrifugal_pump type. The type is defined once. The instances differ only in their tag bindings, commissioned date, and notes. Adding a third pump requires only a new instance entry.

historian

The historian section configures the time dimension available to the Historian agent. It defines the default look-back window for trend queries and any historian-specific connection details if the time series source is separate from the real-time protocol surface.

Key	Type	Required	Description
default_lookback_hours	integer	yes	Default time window for trend queries when no explicit window is specified. Recommended: 24.
max_lookback_days	integer	yes	Maximum historical window the Historian will query. Prevents unbounded history requests.

resolution_seconds	integer	no	Preferred data resolution for history queries in seconds. Default: adapter-native resolution.
source	string	no	If historical data comes from a dedicated historian rather than the real-time adapter, specify the aggregator server name here. Default: same adapter as real-time data.

```
historian:
  default_lookback_hours: 24
  max_lookback_days: 90
  resolution_seconds: 60
```

aggregator.json

The aggregator configuration defines the protocol surface available to the agent layer. It is a JSON array of server definitions, where each entry represents one logical protocol server instance.

A logical server instance is not necessarily a separate physical server. The same protocol server can appear multiple times in the aggregator config, each time scoped differently. This is how a single MQTT broker serving an entire facility is divided into process-area-specific surfaces for the agent layer.

Server definition schema

Key	Type	Required	Description
name	string	yes	Logical name for this server instance. Referenced in topology tag bindings and in agent tool calls.
url	string	yes	MCP server endpoint URL. SSE endpoint for streaming servers.
include_tools	list	no	List of tool names to expose from this server. Tools not listed are not available to agents. Omit to expose all tools.

default_args	object	no	Default parameter values applied to every tool call on this server. Used to scope servers to a section of the protocol surface without modifying the adapter.
timeout_ms	integer	no	Tool call timeout for this server in milliseconds. Default: 5000.
description	string	no	Human-readable description of what this logical server represents. Appears in get_server_info responses.

```
[
  {
    "name": "mqtt_intake",
    "url": "http://localhost:8001/sse",
    "include_tools": ["read_tag", "read_tag_history", "discover_tags"],
    "default_args": {
      "topic_filter": "Plant/WTP/Pump/RawWater_*"
    },
    "description": "Raw water intake pump data"
  },
  {
    "name": "mqtt_treatment",
    "url": "http://localhost:8001/sse",
    "include_tools": ["read_tag", "read_tag_history", "discover_tags"],
    "default_args": {
      "topic_filter": "Plant/WTP/Treatment/*"
    },
    "description": "Treatment process data"
  },
  {
    "name": "mqtt_distribution",
    "url": "http://localhost:8001/sse",
    "include_tools": ["read_tag", "read_tag_history", "discover_tags", "write_tag"],
    "default_args": {
      "topic_filter": "Plant/WTP/Pump/Treated_*"
    },
    "description": "Treated water distribution pumps. Write access enabled."
  }
]
```

write_tag appears only in mqtt_distribution. Intake and treatment servers expose read and discovery tools only. The include_tools list is the primary write access control mechanism at the deployment layer. This is separate from topology writable flags, which define what is physically writable. Both must permit a write for it to succeed.

The Topology Builder

Hand-authoring topology.yaml for a real facility is not a practical workflow. The smallest industrial plant has thousands of tags. A facility built over 20 years by multiple contractors may have tens of thousands. The FieldWorks topology builder is the onboarding tool that makes FieldWorks deployable against a real plant without hand-authoring anything.

The topology builder is an AI-assisted discovery tool that runs through the chat interface. It scans the configured protocol surfaces, infers equipment types and process areas from what it finds and builds the plant topology through a guided conversation with the deployer. The output is a review artifact, a draft topology.yaml, and a committed knowledge graph in LadybugDB.

It is a framework component. It is designed to be protocol-agnostic and plant-type-agnostic. The same tool that onboards a water treatment plant onboards a refinery, a brewery or a power station.

How it works

The deployer starts the topology builder from the normal chat interface with a natural language instruction.

```
Browse this broker and build my plant model
```

The interface shifts into topology builder mode. The diagnostic UI slides out. The topology builder expands to fill the screen. A chat panel on the left, a node visualization panel on the right. The plant materializes in the visualization as discovery progresses.

The builder works through four stages.

Stage 1 — Template selection

Before discovery begins, the builder presents a template library. Templates are pre-built plant type definitions that provide process area structure, expected equipment types, and fault mode libraries for known facility categories. Selecting a template gives the builder a scaffold to map discovered equipment into.

Templates are optional. The deployer can skip template selection and start discovery from scratch. In that case the builder infers process area structure from equipment type groupings and asks the deployer to confirm the process flow.

Official templates are maintained by the FieldWorks project. Community templates are contributed by SIs and operators who have deployed against specific plant types and want to share the starting point. Both are available in the template library.

The template library is a community artifact. An SI who builds a high-quality water treatment template and contributes it back is giving every subsequent water treatment deployment a faster, better start. Over time the template library becomes a repository of industrial domain knowledge — plant structures, equipment types, and fault mode descriptions written by people who have actually maintained that equipment.

Stage 2 — Discovery

The builder connects to the configured protocol adapters through the aggregator and walks the available protocol surfaces. For MQTT it calls `get_full_topic_tree` to retrieve the full topic hierarchy. For OPC-UA it calls `get_node_tree` to traverse the address space. For multi-protocol deployments it scans all configured adapters and cross-references findings.

As topics and nodes are discovered, the builder infers equipment instances using naming pattern analysis. A cluster of topics under `Plant/WTP/Pump/RawWater_01/` — Flow, Pressure, Running, Power — is recognized as a centrifugal pump instance named `RawWater_01`. The inference is probabilistic. Each inferred binding is assigned a confidence score.

During discovery, nodes appear in the visualization panel as they are found, organized into process area columns. The deployer watches the plant model materialize in real time. The confidence-based node sizing makes the uncertain inferences immediately visible. Fully confident nodes render at full size, inferred nodes smaller, suspect nodes smaller still.

```
// Example discovery output – chat panel
MQTT scan complete. 38 topics discovered.
10 equipment instances found across 3 process areas.
All instances verified at 95% confidence.
Dual-sourced against OPC-UA for cross-validation.
```

Stage 3 — Review and correction

When discovery is complete the builder summarizes what it found and flags items that need deployer input. Flagged items appear as amber or red nodes in the visualization. The deployer resolves them through natural language in the chat panel.

```
// Flagged item example
Plant/WTP/OldBroker/P-Legacy is publishing but doesn't match
any recognized equipment pattern. Is this active equipment
or a decommissioned tag?

// Deployer response
P-Legacy is decommissioned, ignore it

// Builder response
Got it. P-Legacy removed from the model.
```

The deployer can also correct classifications, add notes to specific instances, adjust process area assignments and enrich fault mode descriptions with facility-specific knowledge. The builder updates the model in real time and reflects changes immediately in the visualization. The visualization panel supports filtering by process area. Clicking a column header isolates that area's nodes. At scale, with hundreds of equipment instances across many process areas, this makes the review manageable. The deployer works through one area at a time rather than trying to review the full plant simultaneously.

The status bar at the bottom of the visualization shows the current model state at all times. Instance count, verified count, inferred count, suspect count, flagged items. The deployer knows exactly where they stand before committing.

Stage 4 — Review artifact and commit

When the deployer is satisfied with the model, the builder generates a `topology.yaml` as a review artifact. The YAML appears in the chat panel as a complete, human-readable representation of the inferred topology. The deployer can inspect it, copy it for version control or request changes before committing.

Nothing is written to LadybugDB until the deployer explicitly commits. The commit is a deliberate action. A button in the visualization panel and a confirmation prompt in the chat. This is the same operator-in-the-loop principle that governs the action path in the diagnostic system. The AI proposes. The human decides.

On commit, the builder writes the full topology to LadybugDB using the schema defined in Part IV. The knowledge graph is seeded with the static layer. Facility, process areas, equipment types, instances, attributes, tag bindings and fault modes. The dynamic layer begins accumulating on the first diagnostic session.

The interface returns to the diagnostic UI. The fault status panel in the right sidebar is now populated from the committed topology. The specialist agents have plant context on their first query.

Legacy mode

Real industrial plants are not clean. The waterworks-ai reference implementation includes a clean simulator that produces consistent, well-structured MQTT topics. Real plants built over 20 years by multiple contractors produce something very different.

Legacy mode is a simulator configuration that intentionally produces the kind of protocol surface the topology builder will encounter in real deployments. It is used to test and demonstrate the builder's inference capabilities under realistic conditions.

- Inconsistent naming conventions across contractor eras. The same equipment type named three different ways in different sections of the plant.
- Dead tags still publishing stale values from decommissioned equipment.
- Duplicate signals from different integration projects publishing at different sample rates.
- Engineering units in inconsistent conventions. PSI in one area, bar in another, kPa in a third.
- Tags with no units at all.
- Equipment that was renamed but the old tags were never removed.
- A process area that no longer exists but still has active tags.
- Quality signals set to good on everything regardless of actual sensor state.

The topology builder's handling of legacy mode is the honest test of whether it works against real plants. A builder that only works against clean, well-structured tag trees is a demo tool. A builder that produces a useful topology from a messy, inconsistent protocol surface is a deployment tool.

Legacy mode is the honest test. Clean plants make the topology builder look easy. A plant with three naming conventions, decommissioned tags still publishing, and units in three different standards — that is what the builder has to handle in a real SI engagement.

Framework separation

The topology builder is designed as a framework component from the start. It is not a waterworks-specific feature that will be extracted later. The separation is maintained through a clean interface boundary.

- No plant-specific code in the builder. No hardcoded references to waterworks equipment types or topic structures.
- Protocol agnostic. The builder communicates with the aggregator, not directly with MQTT or OPC-UA adapters. Swap the protocol, the builder does not change.

- LadybugDB schema is the framework schema. The builder writes to the graph using the schema defined in Part IV. That schema belongs to the framework.
- Templates are external files. Template definitions live in a `templates/` directory and conform to the template schema. Community templates are files that meet that schema. The builder loads them at startup.
- The UI component is self-contained. The full UI shift, the visualization panel and the chat integration are a component that can be embedded in any FieldWorks deployment interface.

In the waterworks-ai reference implementation, the topology builder lives in a `fieldworks/topology_builder/` directory with a defined interface boundary. waterworks calls it. The framework owns it.

What the builder enables

The topology builder changes the onboarding story for FieldWorks deployments. The deployment workflow becomes:

- Connect the adapters to the protocol surface.
- Select a template or start from scratch.
- Type: browse this broker and build my plant model.
- Review the discovered topology, correct the flags, commit.
- Ask the first diagnostic question.

An SI familiar with the plant type and the FieldWorks framework can complete this workflow in an afternoon. An SI using a community template for a known plant type can complete it faster. The domain knowledge that would otherwise require weeks of manual topology authoring is contributed by the template and refined by the deployer's corrections.

That is the onboarding story that makes FieldWorks accessible at the scale of the SI channel. The topology builder is not a convenience feature. It is the mechanism that makes deployment practical.

Validation

FieldWorks provides two validators. They serve different purposes and are run at different stages of deployment.

Topology validator

The topology validator checks topology.yaml against the schema defined in this specification. It is a static validator that does not require a running protocol surface.

- All required fields are present at every level of the hierarchy.
- Equipment instance type_id references resolve to defined equipment types.
- Equipment instance process_area_id references resolve to defined process areas.
- Tag bindings in each instance cover all attributes defined on the referenced type.
- Normal range min is less than max for all attributes.
- Fault mode severity values are one of: advisory, warning, critical.
- Affected attributes in fault modes reference attribute IDs defined on the equipment type.
- Historian source, if specified, references a name defined in aggregator.json.

```
fieldworks validate topology.yaml
fieldworks validate topology.yaml --aggregator aggregator.json
```

Pass --aggregator to also validate that all tag bindings reference tag IDs discoverable on the configured adapters. Requires running adapters. Omit for offline schema validation only.

Adapter conformance suite

The adapter conformance suite validates that a protocol adapter meets the requirements of Part I. It is a behavioral test suite that runs against a live adapter instance.

- connect and disconnect complete without error.
- discover_tags returns well-formed tag descriptors with all required fields.
- read_tag returns a valid VQT envelope for a known tag.
- read_tag returns a structured TAG_NOT_FOUND error for an unknown tag.
- read_tag_history returns an ordered array of VQT envelopes for a known tag.
- write_tag returns TAG_NOT_WRITABLE for a tag not in write permissions.
- write_tag returns INVALID_VALUE for a value outside the configured engineering range.
- get_server_info returns all required fields including a complete capabilities list.
- browse or scan returns a non-empty result for a configured protocol surface.
- get_topic_tree or get_node_tree returns a structured hierarchy.

```
fieldworks test-adapter --url http://localhost:8001/sse --protocol mqtt
fieldworks test-adapter --url http://localhost:8002/sse --protocol opcua
```

The conformance suite requires a running adapter with at least one readable tag. For write

Complete Worked Example

The following is the complete plant layer configuration for the waterworks-ai reference deployment. A municipal water treatment plant with three process areas, four equipment types, and twelve equipment instances.

This is the configuration that makes waterworks-ai buildable. With this topology.yaml, the aggregator.json above, and conformant MQTT and OPC-UA adapters, the full agent layer can be brought up and will have complete plant context from first query.

The complete waterworks-ai topology.yaml is available in the FieldWorks GitHub organization. The version here is abbreviated to illustrate structure. The repository version is the authoritative reference.

```
facility:
  name: Riverside Water Treatment Plant
  site_id: wtp-riverside-01
  description: Municipal water treatment, 45 MGD capacity
  timezone: America/Chicago
  units_system: metric

process_areas:
  - id: intake
    name: Raw Water Intake
    description: Raw water pumping from source to treatment.
  - id: treatment
    name: Treatment
    description: Coagulation, filtration, disinfection.
  - id: distribution
    name: Distribution
    description: Treated water pumping to network.

equipment_types:
  - id: centrifugal_pump
    name: Centrifugal Pump
    description: Water movement via impeller rotation.
    attributes:
      - id: discharge_pressure
        name: Discharge Pressure
        units: bar
        normal_range: { min: 2.8, max: 4.2 }
```

Continued on the next page.

```

- id: motor_current
  name: Motor Current
  units: A
  normal_range: { min: 18.0, max: 32.0 }
- id: vibration
  name: Vibration
  units: mm/s
  normal_range: { min: 0.0, max: 4.5 }
- id: bearing_temp
  name: Bearing Temperature
  units: degC
  normal_range: { min: 20.0, max: 75.0 }
- id: run_status
  name: Run Status
  units: ""
  normal_range: { min: 0, max: 1 }
- id: speed_setpoint
  name: Speed Setpoint
  units: "%"
  normal_range: { min: 40.0, max: 100.0 }
  writable: true
fault_modes:
- id: bearing_wear
  name: Bearing Wear
  severity: warning
  affected_attributes: [vibration, bearing_temp, motor_current]
  description: Gradual vibration increase over hours to days, rising
    bearing temperature, slight motor current increase.
- id: cavitation
  name: Cavitation
  severity: warning
  affected_attributes: [discharge_pressure, vibration, motor_current]
  description: Erratic discharge pressure, high-frequency vibration,
    unstable motor current. Causes rapid impeller damage if sustained.

# ... uv_bank, chemical_dosing_pump, filter_bed types follow same pattern

```

Continued on the next page.

```
equipment_instances:
  - id: raw_water_pump_1
    name: Raw Water Pump 1
    type_id: centrifugal_pump
    process_area_id: intake
    commissioned: "2019-03-15"
    notes: Impeller replaced 2024-11-02 following cavitation damage.
    tag_bindings:
      discharge_pressure: Plant/WTP/Pump/RawWater_P01/DischargePressure
      motor_current:      Plant/WTP/Pump/RawWater_P01/MotorCurrent
      vibration:          Plant/WTP/Pump/RawWater_P01/Vibration
      bearing_temp:       Plant/WTP/Pump/RawWater_P01/BearingTemp
      run_status:         Plant/WTP/Pump/RawWater_P01/RunStatus
      speed_setpoint:     Plant/WTP/Pump/RawWater_P01/SpeedSetpoint

  # ... remaining instances follow same pattern

historian:
  default_lookback_hours: 24
  max_lookback_days: 90
  resolution_seconds: 60
```

Modeling Imperfect Plants

Real industrial facilities are not clean. Decades of different contractors, different conventions, different commissioning standards and deferred maintenance produce protocol surfaces that don't match any idealized model. An MQTT broker serving a plant built in stages over 20 years may have three different naming conventions for the same equipment type, sensors that failed and were never removed, tags with no engineering units and quality signals that were never properly configured.

FieldWorks does not pretend this isn't true. The topology model provides four mechanisms for representing reality honestly. A topology that accurately describes a messy plant is more useful than one that pretends the plant is clean.

Optional attribute bindings

Not every instance of an equipment type will have every attribute the type defines. A pump type may define a vibration attribute because most pumps in the facility have vibration sensors. Some don't. The instances without sensors should not be forced to fabricate a binding.

Tag bindings are optional at the instance level. An instance binds the attributes it actually has. The topology validator will warn on missing bindings but will not fail on them. The Specialist receives only the attributes that are bound and reasons with what is available. It notes what is missing rather than silently ignoring it.

```
equipment_instances:
  - id: raw_water_pump_3
    name: Raw Water Pump 3
    type_id: centrifugal_pump
    process_area_id: intake
    tag_bindings:
      discharge_pressure: Plant/WTP/Pump/RawWater_P03/DischargePressure
      motor_current:      Plant/WTP/Pump/RawWater_P03/MotorCurrent
      run_status:         Plant/WTP/Pump/RawWater_P03/RunStatus
      # vibration, bearing_temp not instrumented on this unit
```

Confidence on tag bindings

Tag bindings carry an optional confidence field that records how the binding was established. This flows directly into Specialist reasoning. A diagnosis built on verified bindings is more reliable than one built on inferred bindings, and the Specialist should say so.

Key	Type	Required	Description
<code>confidence</code>	string	no	How this binding was established. One of: verified, inferred, suspect. Default: verified if omitted.

```
tag_bindings:
  discharge_pressure:
    tag_id: Plant/WTP/Pump/RawWater_P01/DischargePressure
    confidence: verified
  vibration:
    tag_id: Plant/WTP/Pump/RawWater_P01/Vibration
    confidence: inferred
  bearing_temp:
    tag_id: Plant/WTP/Pump/RawWater_P01/BearingTemp_OLD
    confidence: suspect
```

- **verified** — binding was confirmed against commissioning documentation or direct inspection. The tag ID is correct and the signal is reliable.
- **inferred** — binding was proposed by the topology builder based on naming convention analysis. Has not been manually confirmed. Treat with appropriate caution.
- **suspect** — binding is known to be unreliable. The tag exists and is readable but the signal has known quality issues. The Specialist should note this in any diagnosis that uses it.

The expanded tag_bindings syntax with tag_id and confidence is fully compatible with the simple string syntax used in earlier examples. Simple string bindings are treated as confidence: verified. Use the expanded syntax only when confidence needs to be recorded.

Notes on tag bindings

Individual tag bindings accept a notes field for known issues that cannot be expressed in structured fields. This is where institutional knowledge lives.

```
tag_bindings:
  discharge_pressure:
    tag_id: Plant/WTP/Pump/RawWater_P01/DischargePressure
    confidence: verified
    notes: Transmitter replaced 2023-08, reading approximately 0.15 bar
           high until recalibration scheduled for Q3 2026.
  bearing_temp:
    tag_id: Plant/WTP/Pump/RawWater_P01/BearingTemp
    confidence: suspect
    notes: Sensor has been intermittently dropping to 0.0 since 2022.
           Quality signal is unreliable. Treat bad quality as sensor fault
           not equipment fault.
```

Notes on tag bindings are visible to Specialists. An experienced maintenance technician who knows that a pressure transmitter reads high can record that knowledge here. The Specialist incorporates it. Knowledge that would otherwise exist only in someone's head becomes part of the plant model.

Explicit missing attribute handling

The Specialist agent design in Part III addresses directly what happens when a diagnosis encounters missing, inferred, or suspect data. The rules are simple:

- A Specialist that cannot read a bound tag due to quality bad or connection error reports the gap explicitly. It does not silently exclude the attribute from its reasoning.
- A Specialist reasoning against an inferred binding qualifies its conclusion. It may diagnose but it notes the confidence limitation.
- A Specialist reasoning against a suspect binding treats the signal as corroborating evidence only, not primary evidence. A diagnosis does not rest on a suspect signal alone.
- A Specialist asked to diagnose equipment with no vibration binding does not pretend vibration data is unavailable due to a fault. It states that this equipment is not instrumented for vibration and adjusts its diagnostic scope accordingly.

The goal is honest diagnosis. A Specialist that says "bearing wear is possible but I cannot confirm without vibration data, which is not instrumented on this unit" is more useful than one that either ignores the gap or fabricates a conclusion. Operators trust systems that know what they don't know.

What Comes Next

Part II is complete when you have a validated `topology.yaml` and a working `aggregator.json`, and the adapter conformance suite passes against your protocol servers. At that point the plant layer is fully configured. The agent layer has everything it needs to reason about your facility.

Part III covers the agent layer: the fixed agent designs, how Cascade uses the topology to route queries, how Specialists consume equipment type context and fault modes, how Deadband validates before action, how Millwright executes with write access, the audit trail contract, and the trust mode configuration that governs how much autonomy the system operates with.

Part III - The Agent Layer

Overview

The agent layer is where reasoning happens. It sits above the plant layer, receives queries from operators, reads from the protocol surface through the aggregator, and returns findings or executes actions. It never touches the protocol layer directly. It never bypasses the aggregator. It never acts without leaving a record.

The agent layer has five fixed agents. Their names are drawn from industrial vocabulary. That is not an accident. The names carry meaning that people who work in plants will recognize immediately.

- **Cascade** — orchestrates. The entry point for every query. Loads plant context, routes to Specialists, synthesizes findings.
- **Specialist** — diagnoses. One per process area, defined by topology. Knows its area in depth.
- **Historian** — provides time dimension. Consulted when trend context adds value to a diagnosis.
- **Deadband** — validates. Noise or signal? The gate between the diagnostic path and the action path.
- **Millwright** — executes. Write access only. Called when things need to change.

The agent layer has two paths. The diagnostic path runs on every query. The action path runs only when a validated finding warrants intervention. The audit trail runs on both, always, without exception.

The Two Paths

The diagnostic path

The diagnostic path is the path of understanding. It begins when Cascade receives a query and ends when Cascade returns a synthesized finding to the operator. Every query travels the diagnostic path. Most queries go no further.

- Cascade receives the query and loads plant context from `topology.yaml`.
- Cascade determines which process areas are relevant and dispatches the corresponding Specialists.
- Each Specialist reads current tag values through the aggregator and evaluates them against the equipment type definitions and fault mode descriptions in its process area.
- When trend context adds diagnostic value, Cascade dispatches the Historian to read historical data for the relevant tags.

- Cascade synthesizes the Specialist and Historian findings into a coherent response and returns it to the operator.

The diagnostic path is read-only. No tags are written. No state changes. The operator receives a finding and decides what to do with it.

The action path

The action path is the path of intervention. It begins when a diagnostic finding warrants a control action and ends when the outcome of that action is confirmed and logged. Not every finding leads to the action path. Many findings are advisory. The action path is taken only when something needs to change.

- A diagnostic finding identifies a condition that warrants intervention.
- Deadband validates the finding. Is this noise or signal? Does the condition meet the threshold for action given its duration, trend direction, and confidence?
- If Deadband validates the finding as signal, and the action is destructive, operator approval is requested. Destructive actions include setpoint writes, alarm acknowledgements, mode changes, and any operation that cannot be trivially undone.
- The operator approves or denies. In supervised trust mode this step is always present. In assisted mode it is present for destructive actions. In autonomous mode it is bypassed within topology-defined limits, but the operator is notified.
- Millwright receives the approved action, executes it through the aggregator, confirms the outcome, and logs every step.

The action path always produces an audit record regardless of trust mode. An autonomous action that bypassed operator approval is logged as clearly as one that required it. The log is the accountability mechanism. It cannot be disabled.

The Agents

Cascade

Orchestrator. The entry point for every query.

Cascade is always in control, never in a hurry. It does not diagnose. It does not execute. It orchestrates — loading context, dispatching the agents that have the knowledge for the task, and synthesizing what they return into something an operator can use.

Cascade is the only agent the operator interacts with directly. Everything else happens behind it. When an operator asks a question, they are talking to Cascade. When they receive a finding,

it came from Cascade. What Cascade synthesized to produce that finding is the agent layer at work.

What Cascade knows

- The full topology — every process area, every equipment type, every instance, every fault mode.
- Which Specialists exist and which process areas they are responsible for.
- The current trust mode and what it permits.
- The audit trail contract and its logging obligations.

Cascade does not know raw tag values. It does not read from the protocol surface directly. It knows the plant as a model and dispatches agents that know the plant as data.

What Cascade does

- Receives operator queries and determines scope — which process areas are relevant, which agents to dispatch.
- Loads plant context from topology.yaml and injects it into Specialist system prompts.
- Dispatches Specialists in parallel when a query spans multiple process areas.
- Dispatches the Historian when trend context would improve diagnostic quality.
- Synthesizes findings from multiple Specialists into a coherent, operator-facing response.
- Routes validated findings to the action path when Deadband determines intervention is warranted.
- Logs query receipt, dispatch decisions, and response synthesis to the audit trail.

What Cascade does not do

- Read tag values directly from the protocol surface.
- Execute write operations.
- Bypass Deadband on the action path.
- Make diagnostic judgments that belong to a Specialist.
- Return findings without logging them.

Specialist

Diagnostician. One per process area, defined by topology.

Specialists are the diagnostic intelligence of the framework. Each one is responsible for a single process area. Each one is instantiated from the topology at startup, not hardcoded. Change the topology, change the Specialists. Add a process area, get a new Specialist. A Specialist knows its process area the way a good field engineer knows their section of the plant. It knows the equipment, the normal operating ranges, the fault mode patterns, the tag bindings, and the confidence level of each binding. It reads current values and evaluates them against that knowledge.

What a Specialist knows

- The equipment instances in its process area — types, attributes, tag bindings, confidence, notes.

- The normal operating ranges for every instrumented attribute on every instance.
- The fault mode descriptions for every equipment type in its area.
- Which tag bindings are verified, inferred, or suspect.
- Which attributes are missing from which instances, and why.

This knowledge is injected into the Specialist system prompt at instantiation time by Cascade, built from topology.yaml. The Specialist does not query the topology at runtime. It receives its context fully formed.

What a Specialist does

- Reads current tag values through the aggregator for all bound tags in its process area.
- Evaluates current values against normal operating ranges.
- Reasons against fault mode descriptions to identify conditions that match known failure patterns.
- Qualifies findings based on binding confidence — verified findings are stated clearly, inferred findings are flagged, suspect bindings are noted.
- Notes missing attributes explicitly rather than silently excluding them from the diagnosis.
- Returns a structured finding to Cascade with condition assessment, supporting evidence, confidence level, and recommended action if warranted.

What a Specialist does not do

- Read tags outside its assigned process area.
- Call write operations.
- Access historical data directly. It requests historical context through Cascade, which dispatches the Historian.
- Make action decisions. It returns findings. Deadband and the action path handle what happens next.

Specialists are dynamic. The topology defines how many exist and what they know. The framework instantiates them at startup by calling `build_specialist_prompt()` for each process area defined in topology.yaml.

```
def build_specialist_prompt(process_area, equipment_instances, equipment_types):
    # Load all instances belonging to this process area
    # For each instance, load type definition, attribute schema,
    # fault modes, tag bindings with confidence and notes
    # Build a system prompt that gives the Specialist complete
    # knowledge of its area without requiring runtime topology queries
    ...
```

The output is a system prompt that is the Specialist's complete operating context. A water treatment plant with three process areas produces three Specialists. A refinery with twelve process areas produces twelve. The framework code does not change.

Historian

Time dimension specialist. Consulted on demand.

Most diagnostic questions are answered by current values. A pump running outside its normal pressure range is a finding right now. The Historian is not needed for that.

Some diagnostic questions require time. Is the vibration increasing or has it been elevated for weeks? Did the pressure drop suddenly or has it been drifting? Is this the first time this fault has appeared or a recurring pattern? The Historian answers those questions.

Cascade decides when to dispatch the Historian. Not every query warrants historical context. Dispatching it unnecessarily adds latency and cost. Cascade makes the call based on whether the query or the Specialist findings suggest that trend data would change the diagnosis.

What the Historian knows

- The historian configuration from topology.yaml — default lookback window, maximum lookback, resolution, data source.
- Which tags are bound to which equipment instances and attributes.
- Normal operating ranges for the attributes it is asked to analyze.

What the Historian does

- Reads historical tag data through the aggregator using `read_tag_history`.
- Analyzes trend direction, rate of change, and duration of exceedance.
- Identifies recurring patterns — has this fault mode appeared before, how often, under what conditions.
- Provides time-stamped context that sharpens or qualifies Specialist findings.
- Returns a structured time dimension finding to Cascade for synthesis.

```
// Session opens – Cascade receives the query
{"ts":"2026-06-02T16:03:49.728265+00:00","event":"session_start",
 "session_id":"fd01fb5e...", "model":"multi/claude-sonnet-4-6",
 "user_message":"give me a health overview of the plant"}

// Three specialists fan out simultaneously – note near-identical timestamps
{"ts":"2026-06-02T16:03:51.678591+00:00","event":"tool_call",
 "tool":"mqtt__get_full_topic_tree", "args":{}, "specialist":"treatment"}
{"ts":"2026-06-02T16:03:51.708151+00:00","event":"tool_call",
 "tool":"mqtt__get_full_topic_tree", "args":{}, "specialist":"distribution"}
{"ts":"2026-06-02T16:03:51.727619+00:00","event":"tool_call",
 "tool":"mqtt__get_full_topic_tree", "args":{}, "specialist":"intake"}

// Results return as each specialist completes
```

```

{"ts":"2026-06-02T16:03:51.805003+00:00","event":"tool_result",
 "tool":"mqtt__get_full_topic_tree","result":"Plant\n  WTP...",
 "specialist":"distribution"}

// Historian dispatched for time dimension
{"ts":"2026-06-02T16:03:52.140873+00:00","event":"tool_call",
 "tool":"influxdb__query",
 "args":{"flux_query":"from(bucket: \"waterworks\")..."},
 "specialist":"historian"}

// Cascade synthesizes and returns – session closes
{"ts":"2026-06-02T16:04:30.735441+00:00","event":"response",
 "text":"# Plant Health Overview – Synthesized Diagnostic Report..."}

```

What the Historian does not do

- Diagnose independently. It provides time context to findings that originate from Specialists.
- Read current values. That is the Specialist's job.
- Initiate action path requests. It returns findings to Cascade.

The Historian is a peer to the Specialists, not a subordinate. Cascade dispatches it with the same authority as a Specialist. The distinction is scope. Specialists own a process area in the present, the Historian owns the time dimension across all areas.

Deadband

Validator. The gate between diagnosis and action.

Deadband is named for one of the most fundamental concepts in process control. A deadband is a range around a setpoint within which no corrective action is taken. Small deviations are ignored. Only deviations that exceed the band trigger a response. This prevents the control system from chasing noise.

The Deadband agent does exactly this for the action path. It receives a diagnostic finding and asks, is this noise or signal? A single reading outside normal range is not necessarily a problem. A sustained exceedance with a deteriorating trend and high-confidence instrumentation is. Deadband knows the difference.

What Deadband evaluates

- **Duration** — how long has the condition been present? A transient spike is different from a sustained deviation.

- **Trend direction** — is the condition stable, improving, or deteriorating? A deteriorating trend with sustained exceedance is more urgent than a stable one at the same level.
- **Confidence** — how reliable is the instrumentation supporting the finding? A finding based on verified bindings and good quality signals carries more weight than one built on inferred bindings or uncertain quality.
- **Fault mode match** — does the pattern match a known fault mode description, or is it anomalous without a recognized pattern? Known fault modes with established diagnostic heuristics are more actionable.
- **Severity** — what severity did the equipment type assign to this fault mode? Advisory, warning, and critical have different thresholds.

Deadband outcomes

- **Noise** — condition does not meet the threshold for action. Finding is returned to Cascade as advisory. Logged. No action path.
- **Signal** — condition meets the threshold. If the proposed action is non-destructive, it may proceed to Millwright per trust mode rules. If destructive, operator approval is requested first.

Deadband does not decide what action to take. It decides whether the finding warrants action at all. The action itself is proposed by the Specialist finding and executed by Millwright. Deadband is the gate, not the planner.

Deadband has a destructive action threshold. Any operation that changes control system state in a way that cannot be trivially undone is destructive. The threshold is intentionally broad. When in doubt, require approval.

- Setpoint writes — any write to a control setpoint.
- Mode changes — automatic to manual, run to standby, and similar state transitions.
- Alarm acknowledgements — clearing an active alarm from the system.
- Permissive overrides — any action that bypasses an interlock or safety permissive.
- Inhibits — suppressing an alarm or protection function.

Non-destructive operations such as reading values, generating reports or sending notifications, do not require approval regardless of trust mode.

Millwright

Executor. Write access only. Called when things need to change.

A millwright is the tradesperson who installs, maintains, and repairs industrial machinery. They are the person who actually turns the wrench. In the agent layer, Millwright is the only agent that touches the control system. Everything else observes and reasons. Millwright acts.

Millwright is not called often. Most queries are diagnostic. Millwright is called when a validated finding has been approved for action and something in the plant needs to change. When it is called, it acts deliberately, confirms the outcome, and documents everything.

What Millwright knows

- The approved action — what tag, what value, what units, which operator approved it, what reason was given.
- The topology write permissions for the target tag — what the hard limits are, what ramping is configured.
- The current value of the target tag before the write.
- The audit trail contract — what it must log and when.

What Millwright does

- Reads the current value of the target tag before acting.
- Validates the proposed write value against topology hard limits. If the value exceeds the limit, the action is rejected regardless of who approved it.
- Executes the write through the aggregator using `write_tag` with `operator_id` and reason.
- Ramps to the target value when a ramping profile is configured, rather than stepping directly.
- Reads the tag value after the write to confirm the change took effect.
- Logs pre-write value, write action, post-write value, `operator_id`, reason, and outcome to the audit trail.
- Returns a structured execution report to Cascade.

What Millwright does not do

- Initiate actions. It receives approved actions from the action path. It does not decide on its own that something should change.
- Override topology hard limits. Not for any reason. Not at any trust mode. The limits are absolute.
- Execute without logging. Every Millwright action produces an audit record.
- Assume success. It always confirms the outcome by reading back the written value.

Millwright's constraint is not a limitation of the framework. It is a design principle. The agent that executes is the most constrained agent in the system. Cascade has broad context. Specialists have deep knowledge. Deadband has judgment. Millwright has one job. Execute what it has been given, within the limits it has been set, and document what happened.

Trust Mode

Trust mode governs how much autonomy the agent layer operates with. It is a deployment configuration, not a runtime toggle. Changing trust mode is a deliberate decision made by the facility, not something that happens automatically or in response to a query.

There are three trust modes. They form a ladder. Each level extends the autonomy of the previous one in a specific, bounded way. What never changes at any level: topology hard limits, the audit trail, and the requirement that Deadband validates before any action path proceeds.

Supervised

Every action path step requires operator approval. Deadband validates the finding and presents it to the operator with a proposed action. Nothing executes until a human says yes.

- Diagnostic path runs without approval.
- Every action path action requires explicit operator approval before execution.
- Millwright executes only on approved actions.
- Topology hard limits enforced.
- Full audit trail.

Supervised is the appropriate mode for initial deployment, unfamiliar facilities, and any situation where the cost of an incorrect autonomous action is high. It is also the right mode when operator trust in the system is still being established.

```
// Node tables – static layer
CREATE NODE TABLE Facility (id STRING PRIMARY KEY, name STRING, ...)
CREATE NODE TABLE ProcessArea (id STRING PRIMARY KEY, name STRING, ...)
CREATE NODE TABLE EquipmentType (id STRING PRIMARY KEY, name STRING, ...)
CREATE NODE TABLE Equipment (id STRING PRIMARY KEY, name STRING, notes STRING, ...)
CREATE NODE TABLE Attribute (id STRING PRIMARY KEY, units STRING,
    normal_range_min DOUBLE, normal_range_max DOUBLE,
    writable BOOLEAN, requires_confirmation BOOLEAN,
    write_limit_min DOUBLE, write_limit_max DOUBLE)
CREATE NODE TABLE TagBinding (id STRING PRIMARY KEY, tag_id STRING,
    confidence STRING, notes STRING)
CREATE NODE TABLE FaultMode (id STRING PRIMARY KEY, name STRING,
    severity STRING, description STRING)

// Relationship tables – static layer
CREATE REL TABLE CONTAINS_AREA (FROM Facility TO ProcessArea)
CREATE REL TABLE CONTAINS_EQUIPMENT (FROM ProcessArea TO Equipment)
CREATE REL TABLE IS_TYPE (FROM Equipment TO EquipmentType)
CREATE REL TABLE DEFINES_ATTRIBUTE (FROM EquipmentType TO Attribute)
CREATE REL TABLE HAS_FAULT_MODE (FROM EquipmentType TO FaultMode)
CREATE REL TABLE AFFECTS (FROM FaultMode TO Attribute)
CREATE REL TABLE BINDS_ATTRIBUTE (FROM Equipment TO TagBinding,
    attribute_id STRING)
```



```
CREATE REL TABLE BINDING_OF (FROM TagBinding TO Attribute)
CREATE REL TABLE FEEDS_INT0 (FROM ProcessArea TO ProcessArea,
    description STRING)
```

Assisted

The system recommends. The operator decides. Advisory findings are surfaced automatically. Actions that Deadband validates as warranted are presented to the operator as recommendations with proposed actions and supporting evidence. The operator approves or denies.

- Diagnostic path runs without approval.
- Advisory findings are surfaced automatically without operator prompt.
- Destructive actions require operator approval.
- Non-destructive actions may execute automatically.
- Topology hard limits enforced.
- Full audit trail.

Assisted is the mode most facilities will operate in for most of their deployment lifetime. The system adds intelligence without removing the operator from control decisions. It surfaces things the operator might have missed and makes it easy to act on them.

Autonomous

Within topology-defined limits, Millwright executes without per-action approval. The operator is notified after execution. The system acts; the operator reviews.

- Diagnostic path runs without approval.
- Advisory findings are surfaced automatically.
- Actions within topology write permissions and hard limits execute without per-action approval.
- Operator is notified of autonomous actions in real time.
- Topology hard limits enforced. Always. Without exception.
- Full audit trail. Autonomous actions are logged as clearly as supervised ones.

Autonomous mode is appropriate for well-characterized facilities with high-confidence topology models and a track record of reliable system behavior. It is not the default. It is earned through demonstrated performance in supervised and assisted modes.

The trust mode ladder is explicitly modeled on the way experienced operators extend trust to new equipment or new personnel. You supervise first. You assist as confidence grows. You step back when the track record warrants it. The system earns autonomy. It is not granted by default.

Trust mode configuration

Trust mode is set in the aggregator configuration at the deployment level.

```
{
  "trust_mode": "assisted",
  "trust_mode_notes": "Elevated from supervised 2026-04-01 after 90-day supervised evaluation. Reviewed and approved by plant manager.",
  "servers": [ ... ]
}
```

Field	Type	Required	Description
<code>trust_mode</code>	string	yes	One of: supervised, assisted, autonomous.
<code>trust_mode_notes</code>	string	no	Human-readable record of why this mode was set and when. Logged to audit trail on startup.
<code>autonomous_notify_channel</code>	string	no	Notification endpoint for autonomous action alerts. Required if <code>trust_mode</code> is autonomous.

The Audit Trail

The audit trail is not a feature. It is infrastructure. It is as foundational to FieldWorks as the protocol layer. A deployment without a functioning audit trail is not a FieldWorks deployment.

Every session is logged from first query to final response. Every tool call and its result is logged. Every agent dispatch is attributed. Every operator approval or denial is logged. Every Millwright execution is logged. Every trust mode configuration is logged on startup. The log is append-only. It cannot be edited. It cannot be cleared by any agent.

The audit-mcp server

The audit trail is implemented by the audit-mcp server, a required component of every FieldWorks deployment. It exposes a write-only logging interface to the agent layer and a read interface for compliance review and operator inspection.

The audit-mcp server is not optional. The agent layer will not operate without it. Cascade verifies audit-mcp availability at startup and refuses to proceed if it is unreachable.

- Append-only. No agent can modify or delete existing log entries.
- Structured JSONL. One record per line, one file per deployment. Human-readable and machine-parseable.
- Timestamped. Every entry carries a UTC timestamp with millisecond precision.
- Attributed. Every entry carries the agent that produced it and, where applicable, the operator identity.
- Durable. The audit-mcp server persists logs to disk. In-memory-only deployments are not conformant.

Audit record schema

Every record in the audit log is a JSON object on a single line. The schema has two tiers. Fields that every conformant FieldWorks deployment produces today and fields that are planned additions as the framework matures. As the action path is implemented and the agent layer formalizes, the planned fields become standard fields and the distinction disappears.

Current fields produced by every conformant deployment.

Field	Type	Required	Description
<code>ts</code>	string	yes	ISO 8601 UTC timestamp with microsecond precision. e.g. 2026-06-02T16:03:49.728265+00:00
<code>event</code>	string	yes	Event type. One of <code>session_start</code> , <code>tool_call</code> , <code>tool_result</code> , <code>response</code> , <code>incident</code> . See event types below
<code>session_id</code>	string	yes	UUID. Groups all records from a single query session. Set at <code>session_start</code> and propagated to all records in the session.

specialist	string	no	The specialist agent that produced this record. One of historian, cascade, deadband, millwright or any process area id defined in topology.yaml. Present on tool_call and tool_result records in multi-agent deployments
model	string	no	Model identifier. Present on session_start. Format: mode/model-name, e.g. multi/claude-sonnet-4-6

Planned fields added as the framework matures.

Field	Type	Required	Description
event_id	string	yes	UUID. Unique identifier for this audit record.
process_area_id	string	no	Relevant process area, if applicable.
equipment_instance_id	string	no	Relevant equipment instance, if applicable.
operator_id	string	no	Operator identity, present for approval events and write executions.
trust_mode	string	yes	Trust mode active at the time of this event.

The two tiers will merge as the framework develops. A deployment running only the diagnostic path today produces the current fields. When the action path is built, trust_mode and operator_id become standard fields on the relevant records. Existing log consumers will not break. The schema is additive.

Event types

session_start

Logged once per session when Cascade receives an operator query. Contains the full user message and model identifier.

```
{
  "ts": "2026-06-02T16:03:49.728265+00:00",
  "event": "session_start",
  "session_id": "fd01fb5e-67b1-4528-83bc-ceeda3857ac4f",
  "model": "multi/claude-sonnet-4-6",
  "user_message": "give me a health overview of the plant"
}
```

tool_call

Logged each time a specialist calls a tool through the aggregator. In multi-agent mode, multiple tool_call records with different specialist value may share the same timestamp. This indicates parallel fan-out across specialists.

```
{
  "ts": "2026-06-02T16:03:51.678591+00:00",
  "event": "tool_call",
  "session_id": "fd01fb5e-67b1-4528-83bc-ceeda3857ac4f",
  "tool": "mqtt__get_full_topic_tree",
  "args": {},
  "specialist": "treatment"
}
```

Parallel tool_call records with nearly identical timestamps and different specialist values are the audit trail signature of multi-agent fan-out. Three specialists dispatched simultaneously produce three tool_call records within milliseconds of each other.

tool_result

Logged when a tool call returns. Paired with the preceding tool_call by session_id and tool name. Contains the full result payload returned by the protocol adapter or historian.

```
{
  "ts": "2026-06-02T16:03:51.805003+00:00",
  "event": "tool_result",
  "session_id": "fd01fb5e-67b1-4528-83bc-ceeda3857ac4f",
  "tool": "mqtt__get_full_topic_tree",
  "result": "Plant\n WTP\n Pump\n ...",
  "specialist": "distribution"
}
```

response

Logged once per session when Cascade returns the synthesized response to the operator. Contains the full response text. This is the closing record of a session's diagnostic path.

```
{
  "ts": "2026-06-02T16:04:30.735441+00:00",
  "event": "response",
  "session_id": "fd01fb5e-67b1-4528-83bc-ceeda3857ac4f",
  "text": "# Plant Health Overview..."
}
```

The incident record

In addition to the session log, FieldWorks maintains a separate incident index. Each session that produces a finding of diagnostic significance generates an incident record. The incident record is a summary artifact. It provides a high-level index of what was found, on which equipment, at what confidence, without requiring a reader to parse the full session log.

Field	Type	Required	Description
<code>session_id</code>	string	yes	References the full session log for this incident.
<code>ts</code>	string	yes	Timestamp of the session that produced this incident.
<code>user_question</code>	string	yes	The operator query that initiated the session.
<code>equipment</code>	string	no	Comma-separated list of equipment instance IDs implicated in the finding. Empty if no specific equipment was identified
<code>status</code>	string	yes	Diagnostic outcome. One of normal, anomaly detected, fault detected.
<code>confidence</code>	number	yes	Confidence score from 0.0 to 1.0. Reflects the certainty of the finding given instrumentation quality and pattern match strength.
<code>mode</code>	string	yes	Agent mode that produced the finding. One of single, multi.

```
{
  "incidents": [
    {
      "session_id": "fd01fb5e-67b1-4528-83bc-ceeda3857ac4f",
      "ts": "2026-06-02T16:03:49.728265+00:00",
      "user_question": "give me a health overview of the plant",
      "equipment": "Chlorine_01,UV_01,RawWater_02,HighService_02",
      "status": "Anomaly Detected",
      "confidence": 0.78,
      "mode": "multi"
    }
  ]
}
```

Reading a multi-agent session

The following is an abbreviated audit log from a real multi-agent session against the waterworks-ai reference deployment. It illustrates how the parallel fan-out pattern appears in the log and how a session flows from query receipt through specialist dispatch to synthesized response.

The full session spans from 16:03:49 to 16:04:30. 41 seconds from query to synthesized response covering four specialists, multiple InfluxDB queries and cross-system correlation across 33 process data points.

Relationship to the waterworks-ai reference implementation

The waterworks-ai reference implementation uses the audit format defined in this section. It is a single-file JSONL log appended to on every session event, with a companion incidents index updated on every session that produces a diagnostic finding.

The field names and structure defined here are the authoritative contract. The waterworks-ai implementation is conformant. Deployments building on FieldWorks should use the same format.

A note on evolution: the current audit format captures the diagnostic path fully. The action path fields — operator_id, approval events, Millwright write confirmations — will be added as the action path is implemented in the framework. The schema is additive; existing log consumers will not break when new event types are added.

Compliance and retention

The audit trail is the primary compliance artifact for FieldWorks deployments. It provides a complete record of what the AI observed, what it concluded, what it recommended, what the operator decided, and what was executed. This record satisfies audit requirements for process safety management, environmental compliance, and operational excellence programs.

- Log files are append-only and must not be modified after writing.
- Retention period is deployment-specific and should be defined in the facility's data governance policy. Minimum recommended retention is 90 days.
- Log files should be backed up to a separate storage location on a schedule consistent with the facility's data protection requirements.

- The incidents index provides a human-readable summary for compliance review without requiring access to the full session logs.
- Session logs contain full tool results including raw process data. Access to session logs should be restricted to authorized personnel consistent with the facility's data access policy.

The Propose/Approve/Execute Primitive

The Trust Mode section above defines what SUPERVISED mode requires. Every action path step requires operator approval and nothing executes until a human says yes. This section defines the framework-level primitive that makes that requirement enforceable rather than aspirational.

Prior to this primitive, the propose/approve/execute pattern existed only as an application-specific implementation inside the waterworks-ai reference application. A real, working pattern, but not a reusable contract any other deployment could build against. This section extracts that pattern into fieldworks-core.

This is the same category of work as extracting SpecialistMemory and the other memory-layer types into the framework. The example and the primitive are being defined together, for the first time, in this release. Treat this section as a new contract, not a description of long-standing behavior.

How it works

An agent that wants to change something in the physical process does not call a write tool directly. Instead, it proposes the type of action, the target, the proposed value and its rationale. The host application intercepts that proposal before it reaches the underlying MCP server, holds it in a pending state and presents it to the operator.

Nothing executes until the operator responds. An approval releases the proposal for execution. A denial closes it. Either way, the decision is recorded with the same fidelity. A denial is not a lesser event than an approval. It is the operator's judgment being exercised exactly as the architecture intends and the audit trail treats it accordingly.

- The agent proposes. It never writes directly to the protocol layer.
- The host application owns how the proposal reaches the operator either with a UI dialog, a notification or a polling endpoint. The framework does not prescribe transport.
- An approval and a denial are recorded as equally complete, equally detailed events.
- Every proposal, and every decision made on it, is logged to the audit trail with the same rigor described above.

What the framework provides, what the host provides

The framework owns the shape of a proposal, the mechanism for holding it pending and the audit record it produces. The host application owns transport, how the proposal tool is named alongside its other tools and how the agent loop calls into the mechanism.

- **Framework provides:** the proposal data shape, the pending-decision registry, the hash-chained audit log and the canonical tool definition a host can use rather than hand-rolling its own.
- **Host provides:** the transport that carries a decision from operator to system, the namespacing of the proposal tool alongside the host's other tools and the wiring into its own agent loop.

This division mirrors the one in Part I between the protocol layer and the agent layer. The framework defines the contract. The deployment defines the plumbing around it.

The audit record

Two records are kept, for two different purposes.

The compliance record is a hash-chained, append-only log, optionally encrypted, that can be verified for tampering by replaying the chain. This is the record that would be produced in response to a 21 CFR Part 11, FSMA or process safety management audit request. It is not a dashboard or a reporting tool. Its only job is to be a trustworthy, tamper-evident account of what was proposed and what was decided.

The operational record is a queryable log, kept separately, meant for day-to-day reporting and review. What got proposed this week, how often the operator approved versus denied, which equipment generates the most proposals. It is not the compliance record of truth. Conflating the two undersells the rigor the first one is built for and overstates the rigor the second one needs.

Scope boundary

This mechanism covers SUPERVISED mode only. Every write is confirmed by an operator before execution. It does not yet implement dispatch logic for the other rungs of the trust mode ladder:

- **ADVISORY** — the system recommends. It never proposes a write for approval at all.
- **COLLABORATIVE** — writes within a pre-approved normal range execute without confirmation. Writes outside that range still require it.

- **AUTONOMOUS** — the proposal step is skipped entirely and the write is logged with the same fidelity as a supervised one.

IMPLEMENTATION STATUS

SUPERVISED mode is implemented in fieldworks-core as of this release. ADVISORY, COLLABORATIVE, and AUTONOMOUS mode-dispatch logic is not yet implemented anywhere in the framework or in the waterworks-ai reference application. This is a documented gap in the current release, not a claim that the full trust mode ladder is realized end to end.

Part IV - The Memory Layer

Overview

Parts I through III describe a FieldWorks deployment that is capable and correct within a session. The protocol layer gives agents access to plant data. The plant layer gives that data meaning. The agent layer reasons, validates and acts. The audit trail records everything.

What Parts I through III do not provide is memory. Every session starts cold. A Specialist dispatched to diagnose a fault on RawWater_01 knows the topology, knows the current values and knows the fault mode descriptions. It does not know that RawWater_01 threw the same fault three times in the past six weeks, that the operator dismissed the first two events as transient or that the vibration binding on that equipment was flagged as suspect after a transmitter replacement. That knowledge exists in several places, in the audit trail, in the historian, in the operator's head but, nothing carries it forward into the next session.

The memory layer closes that gap. It is a four-store architecture that gives agents persistent, structured access to plant knowledge that accumulates across sessions. The agents get smarter over deployments without anyone having to manually update anything.

The context management work addressed efficiency within a session. The memory layer addresses intelligence across sessions. They are related but distinct problems.

The Four Stores

Each store in the memory architecture serves a specific purpose. They are not interchangeable. The right store for a given query depends on the nature of the question being asked.

LadybugDB - knowledge graph

Structured plant knowledge. Equipment topology, fault causality chains, past incident fingerprints, operator decision patterns. Queryable in Cypher. The static layer describes what the plant is. The dynamic layer accumulates what has happened to it.

DuckDB - analytical layer

Long-horizon cross-equipment analytics. Materializes from InfluxDB on a configurable schedule. The Historian reaches for DuckDB when a query spans weeks or requires cross-equipment joins. InfluxDB when it needs recent trends.

InfluxDB - time-series source

Long-horizon cross-equipment analytics. Materializes from InfluxDB on a configurable schedule. The Historian reaches for DuckDB when a query spans weeks or requires cross-equipment joins. InfluxDB when it needs recent trends.

Specialist memory files - per-specialist persistent notes

Unstructured cross-session knowledge. Per-specialist markdown files that accumulate equipment deviation observations, confidence updates, and recurring pattern flags. Injected into the system prompt at session start.

LadybugDB - The Knowledge Graph

LadybugDB is an embedded, serverless graph database. A property graph model with a Cypher query interface. It runs in-process alongside the FieldWorks application, requires no separate server and is appropriate for air-gapped on-premises deployments. It is MIT licensed and is the active community continuation of the Kuzu project.

The knowledge graph has two layers. The static layer describes the plant as it is configured. The dynamic layer accumulates what has happened to it during operation. Together they make the plant model queryable as a living knowledge base rather than a static configuration file.

The static layer

The static layer mirrors the topology model from Part II. Every node type and relationship in topology.yaml has a corresponding representation in the graph. On first startup, the framework seeds the graph from the topology configuration. From that point forward, the graph is the primary topology store.

TagBinding is a node, not an edge. This is a deliberate design decision. A binding carries properties — confidence, notes, known issues — that would be awkward to express on a relationship. As a node, a binding is a first-class queryable object. The BINDS_ATTRIBUTE and BINDING_OF relationships connect it to equipment and attribute respectively.

The dynamic layer

The dynamic layer accumulates operational history. It is written to by the audit trail pipeline and by specialist agents as they complete sessions. It makes the plant model temporal. Not just what the plant is, but what has happened to it and what decisions have been made about it.

```
// Node tables – dynamic layer
CREATE NODE TABLE Incident (id STRING PRIMARY KEY, ts TIMESTAMP,
    session_id STRING, status STRING, confidence DOUBLE,
    user_question STRING, mode STRING)
CREATE NODE TABLE Observation (id STRING PRIMARY KEY, ts TIMESTAMP,
    tag_id STRING, value DOUBLE, quality STRING, note STRING)
CREATE NODE TABLE OperatorDecision (id STRING PRIMARY KEY, ts TIMESTAMP,
    operator_id STRING, action STRING, reason STRING, outcome STRING)

// Relationship tables – dynamic layer
CREATE REL TABLE INCIDENT_ON (FROM Incident TO Equipment)
CREATE REL TABLE CONSISTENT_WITH (FROM Incident TO FaultMode)
CREATE REL TABLE OBSERVATION_ON (FROM Observation TO Equipment)
CREATE REL TABLE DECISION_ON (FROM OperatorDecision TO Incident)
CREATE REL TABLE PRECEDES (FROM Incident TO Incident,
    days_between DOUBLE)
```

The PRECEDES relationship connects incidents on the same equipment in chronological order. This is what makes recurrence patterns queryable. A Specialist can ask, how many times has this fault appeared on this equipment, how far apart were the events and what did the operator do each time. Without PRECEDES, answering that question requires parsing the audit log. With it, it is a single Cypher traversal.

CONSISTENT_WITH connects an incident to the fault mode it matched. This is the link between the dynamic layer and the static layer. When the Specialist diagnoses a bearing wear fault on RawWater_01, the incident node is connected to the bearing_wear fault mode node. Future sessions can query: which fault modes have appeared on this equipment, and how recently.

Write permission metadata

The Attribute node carries write permission metadata in the static layer. This is the graph representation of the write constraints defined in Part II. Millwright queries the graph before executing a write to confirm the tag is writable, whether operator confirmation is required, and what the hard limits are.

Field	Type	Required	Description
writable	boolean	yes	Whether this attribute accepts writes via write_tag. Default false.
requires_confirmation	boolean	yes	Whether operator approval is required before a write is executed. Applies only when writable is true.

<code>write_limit_min</code>	double	no	Hard lower bound on any write value. Millwright rejects writes below this value regardless of what any agent or operator requests.
<code>write_limit_max</code>	double	no	Hard upper bound on any write value. Millwright rejects writes above this value regardless of what any agent or operator requests.

The hard limits in the graph are the same limits defined in topology.yaml. They are not advisory. Millwright does not override them for any reason at any trust level. The graph is the enforcement layer for write constraints.

Example queries

The following Cypher queries illustrate the kinds of questions the graph makes answerable.

Build a specialist prompt for a process area

Everything a Specialist needs to know about its process area in a single traversal.

```
MATCH (area:ProcessArea {id: 'intake'})-[:CONTAINS_EQUIPMENT]->(e:Equipment)
      -[:IS_TYPE]->(t:EquipmentType)
OPTIONAL MATCH (t)-[:DEFINES_ATTRIBUTE]->(attr:Attribute)
OPTIONAL MATCH (t)-[:HAS_FAULT_MODE]->(fm:FaultMode)
OPTIONAL MATCH (e)-[:BINDS_ATTRIBUTE {attribute_id: attr.id}]>(b:TagBinding)
RETURN area.name, area.specialist_prompt, e.name, e.notes,
       t.name, attr.name, attr.units,
       attr.normal_range_min, attr.normal_range_max,
       b.tag_id, b.confidence, b.notes,
       fm.name, fm.severity, fm.description
ORDER BY e.name, attr.name
```

Fault recurrence on a specific equipment instance

```
MATCH (e:Equipment {id: 'RawWater_01'})<-[:INCIDENT_ON]-(i:Incident)
      -[:CONSISTENT_WITH]->(fm:FaultMode)
OPTIONAL MATCH (i)-[:PRECEDES]->(next:Incident)
RETURN i.ts, fm.name, fm.severity, i.confidence,
       i.status, next.ts AS next_occurrence
ORDER BY i.ts
```

Operator decisions following a fault mode

```
MATCH (fm:FaultMode {id: 'pump_cavitation'})<-[CONSISTENT_WITH]-(i:Incident)
      <-[DECISION_ON]-(d:OperatorDecision)
RETURN d.action, d.outcome, count(*) AS frequency
ORDER BY frequency DESC
```

All writable attributes with hard limits

```
MATCH (e:Equipment)-[:BINDS_ATTRIBUTE]->(b:TagBinding)
      -[:BINDING_OF]->(a:Attribute {writable: true})
RETURN e.name, a.name, b.tag_id,
       a.requires_confirmation,
       a.write_limit_min, a.write_limit_max
ORDER BY e.name
```

DuckDB - The Analytical Layer

DuckDB is an embedded analytical database. It runs in-process, requires no server and is optimized for columnar analytical queries over large datasets. It complements InfluxDB rather than replacing it.

InfluxDB is the right tool for ingesting continuous process data and querying recent time series. It is not designed for complex cross-equipment analytical queries spanning weeks or months. DuckDB fills that gap.

Data pipeline

A configurable pipeline materializes process data from InfluxDB into DuckDB on a schedule. The pipeline runs in the background and does not interrupt real-time operations. The materialization interval is set in the aggregator configuration.

```
# aggregator.json - DuckDB pipeline configuration
{
  "duckdb": {
    "path": "./data/fieldworks.duckdb",
    "sync_interval_minutes": 15,
    "lookback_days": 90
  }
}
```

The DuckDB database holds the materialized process data in a structure optimized for analytical queries. The Historian queries it directly using SQL rather than Flux, which is more expressive for multi-equipment correlation queries.

The Historian's data source hierarchy

The Historian agent selects its data source based on the nature of the query. This distinction is explicit in the Historian's system prompt and is enforced by the available tools rather than left to the model's judgment.

- **InfluxDB** — recent trends, current session context, queries spanning hours to a few days. The Historian defaults here for most queries.
- **DuckDB** — long-horizon analytics, cross-equipment correlations, queries spanning weeks to months. The Historian reaches for `run_correlation` when the query requires it.

The data source distinction is not exposed to the operator. From the operator's perspective, the Historian answers time-dimension questions. Which store it queries to answer them is an implementation detail the Historian manages based on the query scope.

Specialist Memory Files

Specialist memory files are per-specialist markdown files that persist unstructured knowledge across sessions. Each specialist maintains its own memory file. Cascade maintains a shared memory file visible to all agents.

Memory files are written by specialists at the end of a session and read at the start of the next. They are not a log. They are a distillation. A specialist writes only what is worth carrying forward. Observations that change how future sessions should interpret this equipment, confidence updates on tag bindings, recurring patterns that have not been formally captured in the graph.

Memory hierarchy

- **Cascade shared memory** — plant-wide known issues, recent session summaries, operator communication preferences, trust mode history. All specialists read this before starting work.
- **Specialist memory** — process area specific. RawWater_01's binding confidence notes live in the intake specialist's memory, not in Cascade's. Each specialist maintains its own knowledge base about its process area.

- **Historian memory** — pattern knowledge that spans process areas. Recurring fault signatures, seasonal baselines, correlation patterns that the Historian has identified across multiple sessions.

Bootstrap protocol

At session start, each specialist reads its memory file before doing anything else. This is not optional. It is the first step in the specialist's system prompt instruction. The memory file is injected into the system prompt along with the topology context from the graph.

```
# Specialist system prompt – memory bootstrap
BEFORE READING ANY SENSOR DATA, read your memory file at:
  /memories/{process_area_id}/specialist_memory.md

This file contains accumulated knowledge from previous sessions.
Apply it to your diagnostic reasoning before querying current values.
Update it at the end of this session with anything worth carrying forward.
```

What goes in a memory file

Memory files are structured but not rigidly schema-bound. The specialist writes in natural language with consistent section headings. The following sections appear in every well-maintained memory file.

```
# Intake Specialist Memory
Last updated: 2026-06-03T18:45:00Z

### Equipment notes
- RawWater_01: Vibration binding suspect since 2023-08 transmitter replacement.
  Reads consistently 0.8 mm/s high. Weight accordingly.
- RawWater_02: Run status anomaly pattern. Shows non-zero flow when stopped.
  Sensor zeroing deferred – flag for instrumentation team on next inspection.

### Recurring patterns
- Suction starvation on RawWater_01 correlates with high turbidity events.
  Three occurrences in past 90 days, each resolved within 4 hours.
  Check upstream clarifier level when this fault appears.

### Confidence updates
- pump_pressure binding on RawWater_02: downgraded to inferred.
  Transmitter appears to read 0.3 bar low based on cross-check with flow.
```

Operator patterns

- Night shift typically reduces pump speeds proactively during high turbidity.
Context for interpreting low-flow readings between 22:00 and 06:00.

Memory files are not the audit trail. They are not a log of what happened. They are a specialist's working notes — the knowledge worth carrying forward, distilled from sessions. Redundancy with the audit trail is intentional: the audit trail is complete and uneditable, the memory file is curated and current.

The memory-mcp server

The memory layer is exposed to the agent layer through a dedicated MCP server, memory-mcp. It runs alongside the protocol adapters and the audit-mcp server, and is wired into the aggregator configuration.

```
# aggregator.json - memory-mcp server entry
{
  "name": "memory",
  "url": "http://localhost:8006/sse",
  "include_tools": [
    "graph__query",
    "graph__write_incident",
    "graph__write_observation",
    "graph__write_decision",
    "memory__read_specialist",
    "memory__write_specialist",
    "memory__read_cascade",
    "memory__write_cascade",
    "run_correlation"
  ],
  "description": "Knowledge graph, analytical queries, and specialist memory files"
}
```

Tool surface

Field	Type	Required	Description
<code>graph__query</code>	Cypher	yes	Execute a read-only Cypher query against the LadybugDB knowledge graph. Used by specialists to retrieve topology context, fault history, and operator decision patterns.

<code>graph__write_incident</code>	write	yes	Write a new Incident node and connect it to the relevant Equipment and FaultMode nodes. Called by Cascade at the end of sessions that produced a diagnostic finding.
<code>graph__write_observation</code>	write	yes	Write a new Observation node. Called by specialists when they note a tag value or condition worth recording outside the normal audit trail.
<code>graph__write_decision</code>	write	yes	Write a new OperatorDecision node and connect it to the relevant Incident. Called by the action path when an operator approves or denies an action.
<code>memory__read_specialist</code>	read	yes	Read the memory file for a specific specialist. Called at session start as part of the bootstrap protocol.
<code>memory__write_specialist</code>	write	yes	Write or update the memory file for a specific specialist. Called at session end.
<code>memory__read_cascade</code>	read	yes	Read the shared Cascade memory file. Called at the start of every session.
<code>memory__write_cascade</code>	write	yes	Write or update the shared Cascade memory file. Called by Cascade at session end when plant-wide knowledge warrants updating.
<code>run_correlation</code>	DuckDB	yes	Execute a cross-equipment analytical query against the DuckDB materialized dataset. Used by the Historian for long-horizon trend analysis and multi-equipment correlation queries.

Graceful degradation

Memory access degrades gracefully. If memory-mcp is unavailable, sessions still operate. The protocol adapters, aggregator and agent layer continue to function without the memory layer. The only loss is accumulation. Sessions run without knowledge of previous sessions.

This is a deliberate design decision. In industrial settings, uptime is not optional. A memory layer failure must not take the diagnostic capability offline. The memory layer enhances the system. It is not a dependency of the system.

Graceful degradation applies to the memory layer as a whole. Individual tool failures within a session are handled by the agents. A failed `graph__query` is logged and the specialist continues without that context, noting the gap in its findings.

The Memory Lifecycle

A complete session with memory follows a defined sequence.

- Cascade reads the shared memory file via `memory__read_cascade`.
- Cascade dispatches specialists. Each specialist reads its own memory file via `memory__read_specialist` as the first action in its bootstrap.
- Each specialist queries the knowledge graph for topology context, fault history and relevant operator decisions via `graph__query`.
- Specialists read live and historical data from the protocol adapters and InfluxDB. The Historian queries DuckDB via `run_correlation` when long-horizon context is needed.
- Specialists return findings to Cascade. Cascade synthesizes and returns the response to the operator.
- At session end, Cascade writes updated knowledge to the shared memory file via `memory__write_cascade` if anything at the plant level warrants updating.
- Each specialist writes its updated memory file via `memory__write_specialist`, recording any new observations, confidence updates, or patterns identified in this session.
- If the session produced a diagnostic finding, Cascade writes an Incident node to the knowledge graph via `graph__write_incident` and connects it to the relevant Equipment and FaultMode nodes.
- If an operator decision was made on the action path, it is written to the graph via `graph__write_decision` and connected to the Incident.

This lifecycle runs on every session. The accumulation is gradual and continuous. After ten sessions, the specialists know more than they did after one. After a hundred sessions, the knowledge graph holds a meaningful operational history. After a year, the system has institutional knowledge that no individual operator session could have provided.

Phase 2 - Planned Memory Layer Extensions

The four-store architecture described in this part is the current implementation. The following extensions are planned as the framework matures.

Vector store for fault mode retrieval

The current implementation injects all fault mode descriptions for a process area into the specialist system prompt. This works for facilities with a small number of equipment types. At enterprise scale, with hundreds of equipment types and thousands of fault mode descriptions, the context window becomes a constraint.

A vector store over fault mode embeddings allows the framework to retrieve only the fault modes that are semantically similar to what the current sensor data suggests. The specialist receives a focused, relevant set of fault modes rather than the full library. This is a planned addition for enterprise deployments.

Knowledge memory via RAG

Facility documentation — P&IDs, equipment manuals, maintenance procedures, regulatory requirements — contains domain knowledge that no sensor data can provide. A Specialist that can reason against a pump's engineering manual knows things the topology alone cannot tell it.

RAG over facility documentation is planned as a knowledge memory layer. Documents are indexed at deployment time. Specialists query relevant sections when diagnosis requires deeper equipment or process knowledge than the topology provides.

Reactive architecture

The current architecture is query-response. An operator asks a question; the agents answer it. The streaming layer ingests data continuously but agents do not act on it proactively.

The reactive architecture extends this. A streaming processor watches the incoming process data for patterns that match fault mode descriptions in the knowledge graph. When a pattern is detected, it wakes the relevant specialist proactively. The operator is notified before they need to ask.

The reactive architecture requires the memory layer to be in place before it makes sense. An agent waking up proactively without persistent memory just creates noise. With memory, it can filter against known patterns and known history before alerting. That is a planned Phase 2 capability.

The Framework Complete

Parts I, II, III and IV together define a complete FieldWorks deployment. A conformant protocol adapter for every data source. A topology.yaml that describes the facility honestly, including its imperfections. An aggregator.json that scopes the protocol surface to the agent layer. Five agents that observe, reason, validate, and act within defined limits. A four-store memory layer that accumulates plant knowledge across sessions. An audit trail that records everything. A trust mode that the facility controls.

The waterworks-ai reference implementation demonstrates all of it running. It is the spec made real. When the spec is ambiguous, the reference implementation is authoritative.

What FieldWorks does not do is as important as what it does. It does not replace the control system. It does not assume the plant is clean. It does not act without a record. It does not grant autonomy by default. It does not know more than the topology tells it.

What it does is give every facility, regardless of the age of its infrastructure or the protocols it runs, the same thing. A natural language interface backed by agents that know the plant, respect its limits and earn the trust they are given.